Manual de Usuario

Sistema de Apoyo para la Enseñanza del Álgebra Relacional

Trabajo Terminal 2027-A159 · IPN ESCOM

Versión 1.0 · Octubre de 2026

Instituto Politécnico Nacional · Escuela Superior de Cómputo

Presentación

Este manual explica, desde cero, cómo instalar, configurar y usar el **Sistema de Apoyo para la Enseñanza del Álgebra Relacional**. La herramienta permite escribir expresiones de álgebra relacional, observar cómo se tokenizan, analizar su árbol sintáctico, traducirlas a SQL y ejecutarlas sobre una base de datos MySQL local.

Está pensado para estudiantes de Ingeniería en Sistemas Computacionales que cursan sus primeras asignaturas de bases de datos. Por eso, además de describir los botones y pantallas, el documento repasa los conceptos fundamentales del modelo relacional y explica el significado de cada operador.

El sistema fue desarrollado como parte del Trabajo Terminal **2027-A159** de la Escuela Superior de Cómputo del Instituto Politécnico Nacional. Esta es la primera versión: la generación de diagramas entidad-relación y del modelo relacional se reserva para el Trabajo Terminal II.

> ***Nota sobre las capturas.*** *Las imágenes se obtuvieron con una base de datos de demostración llamada* `universidad_escom` *y un servidor MariaDB escuchando en el puerto* `3307`*. En una instalación normal de MySQL el puerto suele ser* `3306`*. El aspecto de la interfaz es el mismo; solo cambian el puerto y, si así se configura, el usuario.*

Control de versiones

Versión	Fecha	Cambios principales
1.0	Octubre de 2026	Primera edición del manual. Conexión a MySQL, diccionario de datos, editor, tokenización, árbol, traducción a SQL, ejecución, historial y catálogo de errores.

Contenido

El manual se organiza en diez capítulos. Los primeros cuatro presentan los conceptos, la instalación y la interfaz; los capítulos 5 y 6 contienen tutoriales y diagnóstico de errores; los últimos cuatro cubren la arquitectura interna, las buenas prácticas, el glosario y los anexos de referencia. Cada entrada termina con el número de página correspondiente.

1. Introducción 3
2. Requisitos y puesta en marcha 7
3. Recorrido por la interfaz 12
4. Fundamentos y sintaxis del álgebra relacional 27
5. Tutoriales guiados 36
6. Errores y diagnóstico 47
7. Cómo funciona el sistema por dentro 52
8. Buenas prácticas, limitaciones y solución de problemas 56
9. Glosario 59
10. Anexos 61

1. Introducción

1.1. Propósito del manual

El objetivo de este manual es que puedas usar el sistema sin ayuda adicional: desde instalar Java y MySQL hasta escribir consultas complejas, interpretar los errores y explicar lo que ocurre en cada etapa de la traducción.

Al terminar de leerlo deberías ser capaz de:

- Conectarte a tu servidor MySQL local y seleccionar una base de datos.
- Consultar el diccionario de datos y el contenido de tus tablas.
- Escribir expresiones correctas de álgebra relacional.
- Entender la tokenización, el árbol sintáctico y la sentencia SQL generada.
- Ejecutar la consulta y leer la tabla de resultados.
- Guardar relaciones en variables `@1`, `@2`, … y reutilizarlas.
- Diagnosticar errores léxicos, sintácticos y de ejecución.
- Explicar la diferencia entre una operación de álgebra relacional y su equivalente en SQL.

1.2. A quién está dirigido

El manual supone que sabes programar en algún lenguaje y que estás comenzando a estudiar bases de datos. No necesitas conocer álgebra relacional, SQL ni MySQL de antemano: los conceptos se introducen conforme se necesitan.

También puede ser útil para profesores que quieran demostrar en clase el proceso completo de traducción y para equipos que retomarán el proyecto en el Trabajo Terminal II.

1.3. Conceptos básicos del modelo relacional

Antes de usar la herramienta conviene tener claros algunos términos. Si ya los conoces, puedes saltar a la sección 1.4.

Base de datos y gestor

Una **base de datos** es una colección organizada de datos relacionados entre sí. Un **gestor de bases de datos** (DBMS, por *Database Management System*) es el programa que almacena, protege y permite consultar esos datos. MySQL y MariaDB son gestores relacionales: organizan la información en tablas y usan SQL como lenguaje de consulta.

En este proyecto, el sistema **no guarda tus datos**: se conecta al MySQL que ya tienes instalado en tu computadora y trabaja sobre las bases de datos que tú hayas creado.

Relación, tupla y atributo

En el modelo relacional, una **relación** es lo que informalmente llamamos tabla. Cada relación tiene:

- Un **esquema**: el nombre de la relación y la lista de sus columnas.
- Un conjunto de **tuplas**: las filas o registros.
- Un conjunto de **atributos**: las columnas, cada una con un nombre y un dominio o tipo de dato.

Término formal	Equivalente informal	Ejemplo
Relación	Tabla	`alumnos`
Tupla	Fila o registro	`(2023630001, Ana María, López, …)`
Atributo	Columna	`nombre`, `nota_media`
Esquema	Estructura de la tabla	`alumnos(boleta, nombre, …)`
Dominio	Tipo de dato permitido	`CHAR(10)`, `DECIMAL(4,2)`, `DATE`

Una relación no debería tener filas duplicadas ni un orden obligatorio. Sin embargo, SQL es un lenguaje práctico y muchas veces permite duplicados y ordenamientos; esta diferencia entre la teoría y la práctica se explica más adelante.

Clave primaria y clave foránea

Una **clave primaria** (*primary key*, PK) es un atributo o conjunto de atributos que identifica de manera única a cada tupla. En `alumnos`, la clave primaria es `boleta`: no puede haber dos alumnos con la misma boleta y el valor no puede ser nulo.

Una **clave foránea** (*foreign key*, FK) es un atributo que hace referencia a la clave primaria de otra relación. En `alumnos`, `id_carrera` es una clave foránea que apunta a `carreras.id_carrera`. Las claves foráneas mantienen la integridad referencial: no debería existir un alumno inscrito en una carrera que no existe.

El diccionario de datos del sistema muestra las claves primarias, las claves únicas y las claves foráneas de cada tabla.

Valor nulo

`NULL` representa la ausencia de un valor: no es cero, no es una cadena vacía y no significa “falso”. En SQL, cualquier comparación directa con `NULL` produce un resultado desconocido (`UNKNOWN`), que no es lo mismo que `FALSE`. Este detalle es importante al escribir condiciones.

1.4. ¿Qué es el álgebra relacional?

El **álgebra relacional** es un conjunto de operaciones formales que se aplican a relaciones y producen nuevas relaciones. Es la base teórica de SQL y permite describir consultas sin preocuparse por cómo se almacenan los datos.

Las operaciones fundamentales que utiliza este sistema son:

- **Selección** (σ): filtra tuplas que cumplen una condición.
- **Proyección** (π): elige un subconjunto de atributos.
- **Reunión** (⋈): combina dos relaciones relacionadas.
- **Unión** (∪), **intersección** (∩) y **diferencia** (−): operaciones de conjuntos.
- **Producto cartesiano** (×): combina cada tupla de una relación con cada tupla de otra.
- **Agrupación y agregación** (γ): agrupa tuplas y calcula totales, promedios, máximos, mínimos o conteos.

Una expresión de álgebra relacional se construye combinando estas operaciones. Por ejemplo:

```
π(nombre, nota_media)(σ(nota_media >= 9)(alumnos))
```

La lectura es de adentro hacia afuera:

1. `σ(nota_media >= 9)(alumnos)` toma la tabla `alumnos` y conserva solo las filas cuya `nota_media` es mayor o igual a 9.
2. `π(nombre, nota_media)(...)` de ese resultado conserva únicamente las columnas `nombre` y `nota_media`.

El resultado es una nueva relación con dos columnas y las filas que cumplen la condición.

1.5. ¿Qué hace este sistema?

El sistema es una aplicación web que traduce expresiones de álgebra relacional a SQL y las ejecuta contra tu MySQL local. El proceso tiene seis etapas:

1. **Conexión:** el usuario proporciona host, puerto, usuario y contraseña de MySQL. El sistema valida la conexión y lista las bases de datos no internas.
2. **Exploración:** el usuario selecciona una base de datos y puede abrir su diccionario de datos: tablas, columnas, tipos, claves, valores predeterminados, comentarios y relaciones.
3. **Análisis léxico:** el texto de la expresión se convierte en una secuencia de **tokens** (símbolos con tipo y posición).
4. **Análisis sintáctico:** un analizador LALR construido con Java CUP verifica que la secuencia respete la gramática y construye un **árbol de sintaxis abstracta** (AST).
5. **Traducción:** cada nodo del árbol sabe generar su equivalente en SQL. Las variables `@1`, `@2`, … se sustituyen por las relaciones guardadas.
6. **Ejecución:** si hay una conexión activa y una base de datos seleccionada, la consulta SQL se ejecuta con JDBC y el resultado se muestra en una tabla. La ejecución es de solo lectura.

El valor pedagógico está en que el sistema **muestra el proceso**, no solo el resultado. Puedes ver los tokens, el árbol, el SQL y los datos reales.

1.6. Alcance de esta primera versión

Esta versión incluye:

- Conexión dinámica al MySQL local del usuario.
- Exploración de bases de datos y diccionario de datos.
- Tokenización visible.
- Árbol sintáctico.
- Traducción a SQL.
- Ejecución de consultas de lectura.
- Historial de relaciones con variables `@n`.
- Catálogo de errores léxicos y sintácticos.
- Editor Monaco con resaltado, tooltips y autocompletado.
- Botones para insertar operadores y ejemplos generados desde la base de datos.

Esta versión **no incluye**:

- Generación de diagramas entidad-relación.
- Generación del modelo relacional.
- Modificación de datos (`INSERT`, `UPDATE`, `DELETE`).
- Creación o alteración de tablas (`CREATE`, `ALTER`, `DROP`).
- Vistas, procedimientos almacenados ni disparadores.

- Inicio de sesión, roles de alumno/profesor ni administración de esquemas.
- Optimización de consultas.

Los diagramas E-R y el modelo relacional se abordarán en el Trabajo Terminal II.

1.7. Convenciones del manual

- Los nombres de botones y campos aparecen **en negritas**, por ejemplo **Traducir y ejecutar**.
- Las expresiones y el código aparecen en `fuente monoespaciada`.
- Los bloques de código con fondo representan texto que puedes copiar y pegar.
- Las notas importantes se marcan con una cita como esta:

> ***Importante.*** *El sistema solo ejecuta consultas de lectura. No uses esta versión para modificar datos. - Las capturas tienen un pie de figura para que puedas localizarlas en la sección correspondiente. - Los símbolos σ, π, ⋈, ∪, ∩, −, × y γ se pueden escribir directamente o sustituir por sus palabras equivalentes (`SELEC`, `PROY`, `JOIN`, etc.). El sistema acepta ambas formas.*

2. Requisitos y puesta en marcha

2.1. Requisitos de hardware y software

Requisito	Mínimo recomendado	Notas
Sistema operativo	Windows 10/11, macOS 12+, Linux con escritorio	El proyecto es Java y funciona en los tres.
Java	JDK 21	El `pom.xml` declara `java.version=21`.
Maven	3.9 o superior	Solo si vas a compilar desde el código fuente.
MySQL	MySQL 8.x o MariaDB 10.5+	MySQL es el gestor objetivo. MariaDB es compatible en las operaciones usadas.
Navegador	Chrome, Edge, Firefox o Safari recientes	Se recomienda Chrome o Edge para Monaco Editor.
Memoria RAM	4 GB libres	8 GB o más para trabajar cómodamente.
Conexión a internet	Necesaria la primera vez	Tailwind CSS, Bootstrap Icons, Google Fonts y Monaco Editor se cargan desde CDN.
Espacio en disco	500 MB libres	Además de lo que ocupen Java, Maven y MySQL.

> ***Sobre la conexión a internet.*** *El servidor de la aplicación no necesita internet para traducir ni para conectarse a MySQL. Es el* ***navegador*** *el que descarga los recursos visuales y Monaco Editor. Si no hay internet, el editor usa un área de texto simple como respaldo: la traducción sigue funcionando, pero se pierden el resaltado, los tooltips y el autocompletado.*

2.2. Instalar Java 21

Windows

1. Descarga un JDK 21 (por ejemplo, Eclipse Temurin, Microsoft Build of OpenJDK u Oracle JDK).
2. Ejecuta el instalador y permite que configure las variables de entorno.
3. Abre una terminal nueva y verifica:

```
java -version
```

Debe aparecer una versión `21.x`.

Ubuntu o Debian

```
sudo apt update
sudo apt install openjdk-21-jdk
java -version
```

macOS con Homebrew

```
brew install openjdk@21
java -version
```

Si `java` no aparece, revisa que la variable `JAVA_HOME` apunte a la instalación del JDK.

2.3. Instalar MySQL

Windows

1. Descarga **MySQL Community Server** desde el sitio oficial de MySQL.
2. Ejecuta el instalador y elige la instalación típica (*Developer Default* o *Server only*).
3. Configura una contraseña para el usuario `root`. **Anótala**: la necesitarás para conectarte desde el sistema.
4. Deja el puerto `3306` salvo que ya esté ocupado.
5. Verifica que el servicio de MySQL esté iniciado.

Ubuntu o Debian

```
sudo apt update
sudo apt install mysql-server
sudo systemctl status mysql
```

Si el servicio no está activo:

```
sudo systemctl start mysql
sudo systemctl enable mysql
```

macOS con Homebrew

```
brew install mysql
brew services start mysql
mysql_secure_installation
```

Para comprobar que el servidor responde:

```
mysqladmin -u root -p ping
```

Debe responder `mysqld is alive`.

2.4. Crear un usuario para la aplicación (recomendado)

Puedes conectarte como `root`, pero para el uso cotidiano es mejor crear un usuario con permisos de solo lectura sobre la base de datos de trabajo. Así evitas que una consulta accidental modifique datos.

Entra a la consola de MySQL:

```
mysql -u root -p
```

Ejecuta algo similar a esto (cambiando la contraseña):

```
CREATE USER 'tt1_user'@'localhost' IDENTIFIED BY 'tt1_pass';
GRANT SELECT, SHOW VIEW ON universidad_escom.* TO 'tt1_user'@'localhost';
FLUSH PRIVILEGES;
```

Con esos permisos el sistema puede leer las tablas y consultar `information_schema` para construir el diccionario. Si necesitas que el usuario vea todas tus bases de datos, puedes otorgar permisos sobre `*.*`, pero para el manual basta una base de datos de práctica.

> ***No uses contraseñas reales en capturas, repositorios ni documentación.*** *En este manual las credenciales son de ejemplo.*

2.5. Obtener y ejecutar el proyecto

El proyecto es una aplicación Spring Boot construida con Maven. Desde la carpeta raíz (`Proyecto_TT1`), ejecuta:

```
mvn clean package -DskipTests
```

Al terminar se genera un archivo JAR en `target/`:

```
target/tt1-algebra-relacional-0.0.1-SNAPSHOT.jar
```

Inicia la aplicación con:

```
java -jar target/tt1-algebra-relacional-0.0.1-SNAPSHOT.jar
```

También puedes ejecutarla directamente con Maven, sin empaquetar:

```
mvn spring-boot:run
```

Cuando veas un mensaje similar a `Tomcat started on port 8080`, abre el navegador en:

```
http://localhost:8080
```

Para detener la aplicación, vuelve a la terminal y presiona `Ctrl+C`.

Cambiar el puerto de la aplicación

El puerto se configura en `src/main/resources/application.properties`:

```
server.port=8080
```

Si el 8080 está ocupado, cámbialo a `8081` y abre `http://localhost:8081`. También puedes pasarlo como argumento al iniciar:

```
java -jar target/tt1-algebra-relacional-0.0.1-SNAPSHOT.jar --server.port=8081
```

Verificar que el puerto esté libre

En Linux o macOS:

```
ss -ltnp | grep 8080
```

En Windows:

```
netstat -ano | findstr :8080
```

2.6. Cargar la base de datos de demostración

El repositorio incluye un script con la base de datos que se usa en las capturas:

```
docs/ejemplos/universidad_escom.sql
```

Para cargarlo desde una terminal:

```
mysql -u root -p < docs/ejemplos/universidad_escom.sql
```

O desde la consola de MySQL:

```
SOURCE /ruta/completa/docs/ejemplos/universidad_escom.sql;
```

El script elimina y vuelve a crear la base `universidad_escom`, con seis tablas:

- `carreras`: catálogo de carreras.
- `alumnos`: datos personales y académicos.
- `profesores`: plantilla docente.
- `materias`: catálogo de materias.
- `inscripciones`: inscripciones y calificaciones.
- `pagos`: pagos de servicios escolares.

Al finalizar, comprueba que exista:

```
SHOW DATABASES;
USE universidad_escom;
SHOW TABLES;
SELECT COUNT(*) FROM alumnos;
```

Debe haber 10 alumnos, 4 carreras, 4 profesores, 6 materias, 18 inscripciones y 7 pagos.

> ***Si ya tienes una base de datos propia**, puedes usarla. Solo asegúrate de que el usuario de MySQL tenga permiso de lectura sobre ella. Las capturas y ejemplos de este manual usan `universidad_escom`.*

2.7. Primera conexión

1. Abre `http://localhost:8080`.
2. En la sección **Conexión a MySQL local**, escribe:
   - **Host:** `localhost` (o `127.0.0.1`).
   - **Puerto:** `3306` (o el puerto que hayas configurado).
   - **Usuario:** tu usuario de MySQL.
   - **Contraseña:** la contraseña de ese usuario.
3. Haz clic en **Conectar**.

Si todo está correcto, el formulario se oculta, aparece la etiqueta **Conectado**, se muestra la versión del servidor y se habilita el selector de **Base de datos**.

2.8. Problemas frecuentes de instalación

Síntoma	Causa probable	Solución
`java: command not found`	Java no está instalado o no está en el PATH.	Instala JDK 21 y abre una terminal nueva. Verifica `JAVA_HOME`.
`mvn: command not found`	Maven no está instalado.	Instala Maven 3.9+ o usa el Maven Wrapper si el proyecto lo incluye.
`Port 8080 was already in use`	Otro proceso usa el puerto.	Cambia `server.port` o detén el otro proceso.
`Access denied for user`	Usuario o contraseña incorrectos, o el usuario no tiene permisos desde ese host.	Revisa credenciales y el host (`localhost` vs `127.0.0.1`).
`Can't connect to MySQL server`	MySQL no está iniciado o el puerto no es el correcto.	Inicia el servicio y verifica el puerto.

Síntoma	Causa probable	Solución
`Unknown database 'universidad_escom'`	No se cargó el script de demostración.	Ejecuta `docs/ejemplos/universidad_escom.sql`.
La interfaz se ve sin estilos	No se descargaron los CDN.	Revisa la conexión a internet o usa el editor de respaldo.
Los tooltips no aparecen	Monaco no cargó o el navegador bloquea scripts.	Usa Chrome/Edge, revisa la consola y la conexión.

3. Recorrido por la interfaz

3.1. Vista general

La pantalla principal se divide en tres zonas:

1. **Encabezado:** nombre del sistema, institución y enlace **Acerca de**.
2. **Conexión a MySQL local:** formulario de conexión o resumen de la conexión activa.
3. **Área de trabajo:** dos columnas. A la izquierda, el editor y los operadores; a la derecha, el proceso de traducción y los resultados.

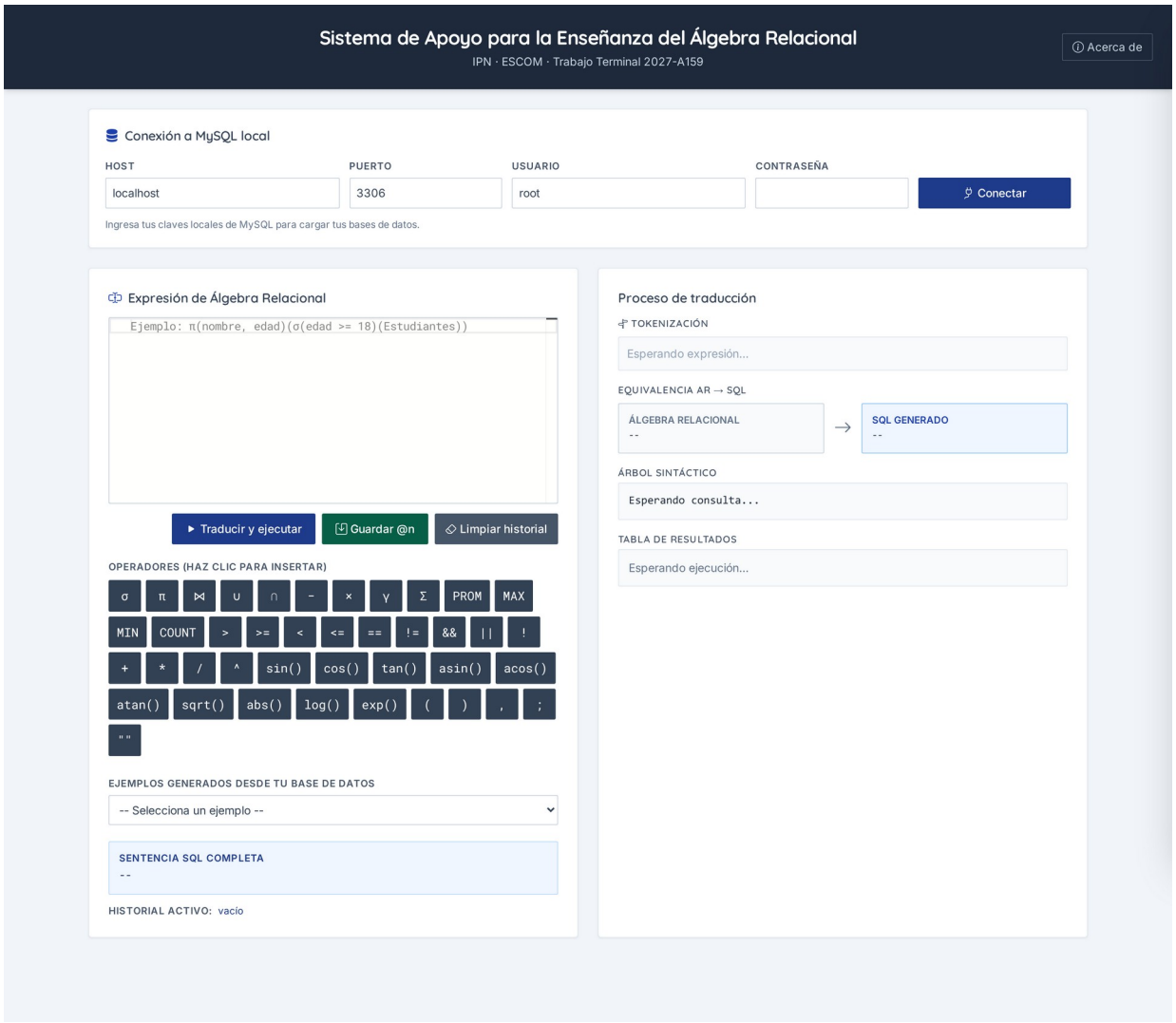


Figura 1. Pantalla principal al abrir el sistema, sin conexión activa.

La distribución es responsiva: en pantallas angostas las dos columnas se apilan verticalmente.

3.2. Panel de conexión

Cuando no hay conexión, el panel muestra cuatro campos y el botón **Conectar**:

Campo	Valor típico	Descripción
Host	`localhost`	Nombre o dirección del servidor MySQL.
Puerto	`3306`	Puerto TCP del servidor.
Usuario	`root` o `tt1_user`	Usuario de MySQL.
Contraseña	(vacía o la tuya)	Contraseña del usuario.

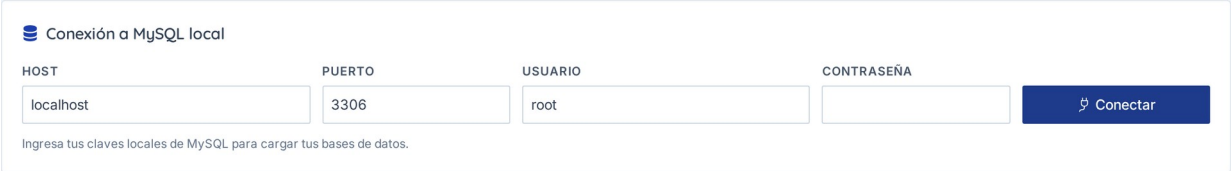


Figura 2. Formulario de conexión a MySQL local.

Al escribir los datos, la contraseña se oculta con puntos:

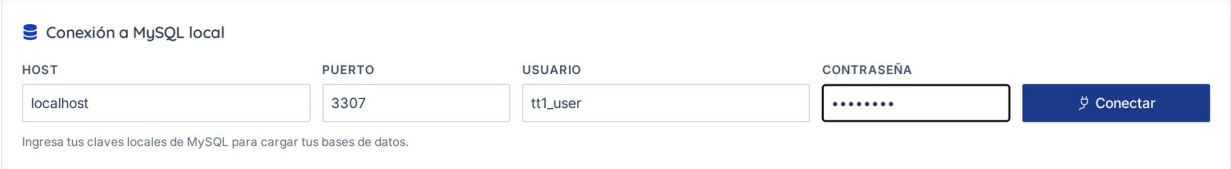


Figura 3. Formulario con los datos de conexión capturados.

Si la conexión falla, el mensaje aparece en rojo debajo del formulario. El sistema no borra los datos capturados para que puedas corregir solo lo necesario.

Conexión establecida

Cuando la conexión es exitosa, el formulario se reemplaza por un resumen:

- Usuario, host y puerto.
- Versión del servidor MySQL/MariaDB.
- Selector de **Base de datos**.
- Botones **Ver diccionario de datos** y **Desconectar**.

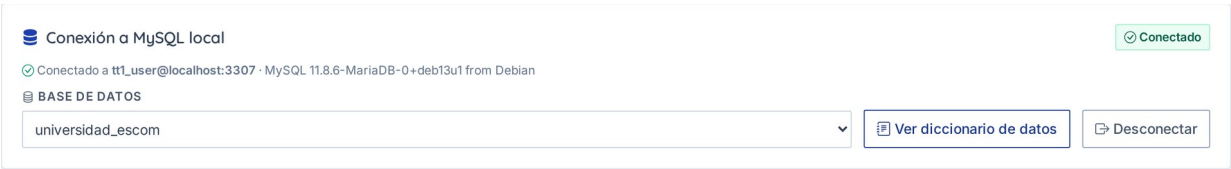


Figura 4. Panel de conexión activa con la base de datos seleccionada.

El selector muestra únicamente bases de datos de usuario: el sistema excluye `information_schema`, `mysql`, `performance_schema` y `sys`. Al cambiar de base de datos se recargan el diccionario y los ejemplos.

Desconectar cierra la conexión lógica del sistema y limpia la lista de bases de datos. No elimina nada de MySQL; solo deja de usar esas credenciales.

3.3. Editor de expresiones

El editor es el corazón de la herramienta. Está basado en **Monaco Editor**, el mismo componente que usa Visual Studio Code.



Figura 5. Editor con una expresión de ejemplo y el panel de operadores.

Características del editor:

- Resaltado de sintaxis para operadores relacionales, funciones, números, cadenas, variables y nombres cualificados.
- Tooltips al pasar el cursor sobre un operador o una palabra clave.

- Autocompletado de operadores (puedes invocarlo con `Ctrl+Espacio`).
- Numeración de líneas oculta para dar más espacio al texto.
- Ajuste de línea activado para expresiones largas.
- Atajo `Ctrl+Enter` (o `Cmd+Enter` en macOS) para traducir y ejecutar sin usar el ratón.

Debajo del editor están los botones:

- **Traducir y ejecutar:** procesa la expresión y, si hay conexión y base de datos, ejecuta el SQL.
- **Guardar @n:** guarda la última expresión traducida como una variable de historial.
- **Limpiar historial:** elimina todas las variables guardadas en el servidor.

3.4. Panel de operadores

Los operadores se insertan con un clic. El panel usa los símbolos siempre que exista uno; cuando el operador no tiene símbolo (o el símbolo no es suficiente), muestra una abreviatura o el nombre de la función.



Figura 6. Panel de operadores. Los botones insertan el símbolo o la plantilla correspondiente.

Al hacer clic en un botón, el texto se inserta en la posición del cursor. Algunos botones colocan el cursor dentro de los paréntesis para que escribas el argumento de inmediato. Por ejemplo, `sin()` inserta la función y deja el cursor entre los paréntesis.

Tooltips

Al pasar el cursor sobre un operador en el editor aparece una tarjeta con:

- El nombre del operador.
- Una descripción breve.
- Un ejemplo de uso.

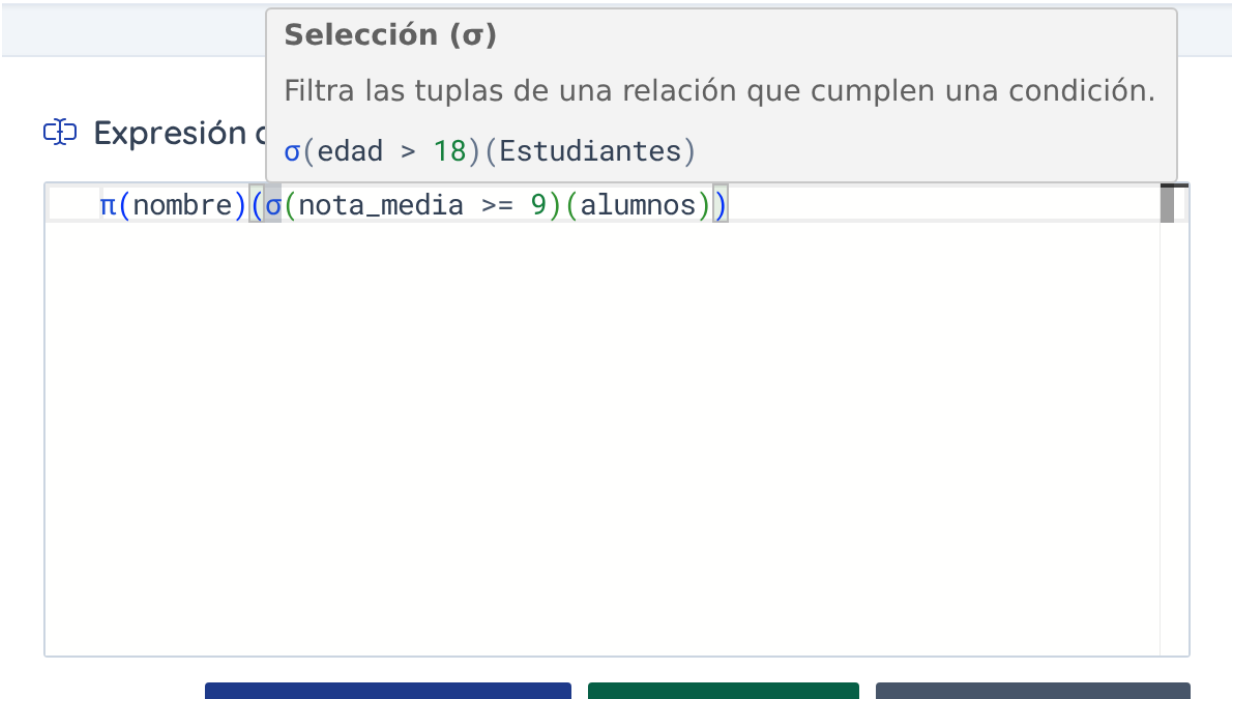


Figura 7. Tooltip del operador de selección (σ) en el editor.

Los tooltips son una ayuda, no una validación: si el ejemplo usa tablas que no existen en tu base, el texto se muestra igual, pero la consulta no se ejecutará.

3.5. Ejemplos generados desde la base de datos

Debajo de los operadores hay un selector de **Ejemplos generados desde tu base de datos**. El sistema toma la primera tabla del diccionario y construye ejemplos automáticos:

- Tabla completa.
- Proyección sobre las primeras columnas.
- Selección con una condición simple.
- Reunión con la segunda tabla (si existe).
- Unión y diferencia con la segunda tabla.
- Agrupación con `COUNT(*)`.

Al elegir un ejemplo, el texto se carga en el editor; **no se ejecuta automáticamente**. Debes revisarlo y pulsar **Traducir y ejecutar**.



Figura 8. Ejemplo de proyección cargado desde el selector.

Los ejemplos son un punto de partida. Pueden incluir columnas que no sean adecuadas para una condición (por ejemplo, comparar una cadena con `0`), así que ajústalos a tu esquema.

3.6. Panel de proceso de traducción

La columna derecha muestra cuatro bloques:

1. **Tokenización:** la lista de tokens reconocidos.
2. **Equivalencia AR → SQL:** la expresión original y el SQL generado.
3. **Árbol sintáctico:** la representación jerárquica de la expresión.
4. **Tabla de resultados:** las filas devueltas por MySQL, con el tiempo de ejecución.

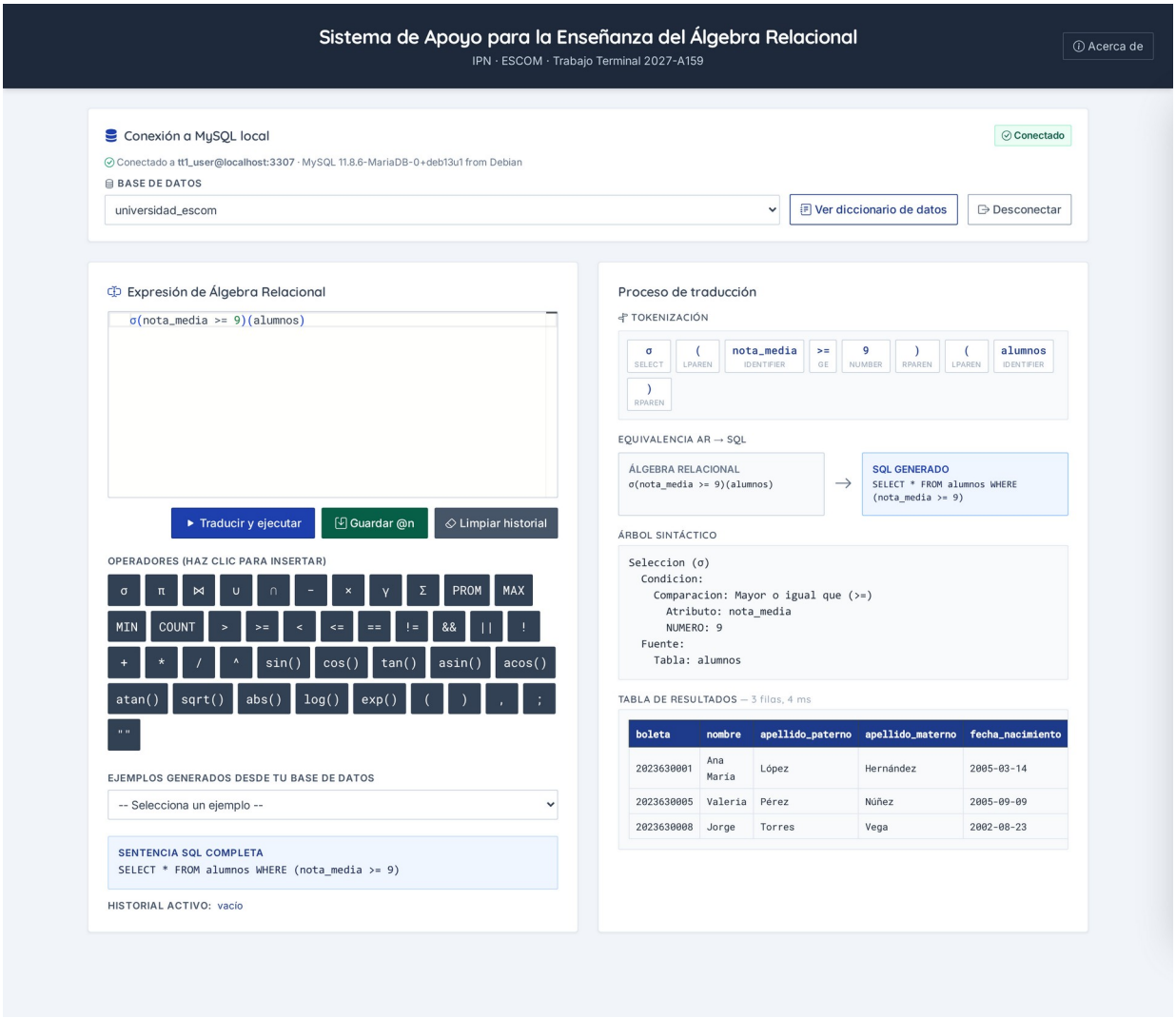


Figura 9. Pantalla completa después de traducir y ejecutar una selección.

Tokenización

Cada token se muestra como una ficha con dos líneas:

- El **lexema**: el texto exacto que aparece en la expresión.
- El **tipo**: el nombre interno del token (`SELECT`, `LPAREN`, `IDENTIFIER`, `GE`, `NUMBER`, etc.).



Figura 10. Detalle de la tokenización de una selección.

En el ejemplo, σ es el lexema y `SELECT` es el tipo interno. No confundas `SELECT` con la palabra SQL: es el nombre que la gramática le da al token de selección. Los tipos se muestran en inglés porque provienen de la definición de Java CUP.

Puedes pasar el cursor sobre una ficha para ver una descripción del tipo de token y su línea/columna.

Equivalencia AR → SQL

El bloque izquierdo muestra la expresión de álgebra relacional; el derecho, el SQL generado. Los alias que empiezan con `_q` son nombres internos que el sistema usa para envolver subconsultas. No los escribas a mano.

Árbol sintáctico

El árbol se muestra como texto con sangría. Cada línea es un nodo; los hijos aparecen más a la derecha. Por ejemplo:

```
Seleccion (σ)
  Condicion:
    Comparacion: Mayor o igual que (>=)
      Atributo: nota_media
      NUMERO: 9
  Fuente:
    Tabla: alumnos
```

La raíz es la operación principal; las hojas son las tablas, los atributos y los literales. Si el árbol es muy grande, el panel tiene desplazamiento vertical.

Tabla de resultados

Si la consulta se ejecutó, se muestran las columnas devueltas y las filas. Los valores `NULL` se muestran en cursiva. Arriba de la tabla aparece el número de filas y el tiempo de ejecución en milisegundos.

Si no hay conexión, la tabla indica que no hubo ejecución. Si la consulta falla en MySQL, el error se muestra en rojo dentro del panel.

3.7. Diccionario de datos

El botón **Ver diccionario de datos** abre un panel lateral con la estructura de la base seleccionada.

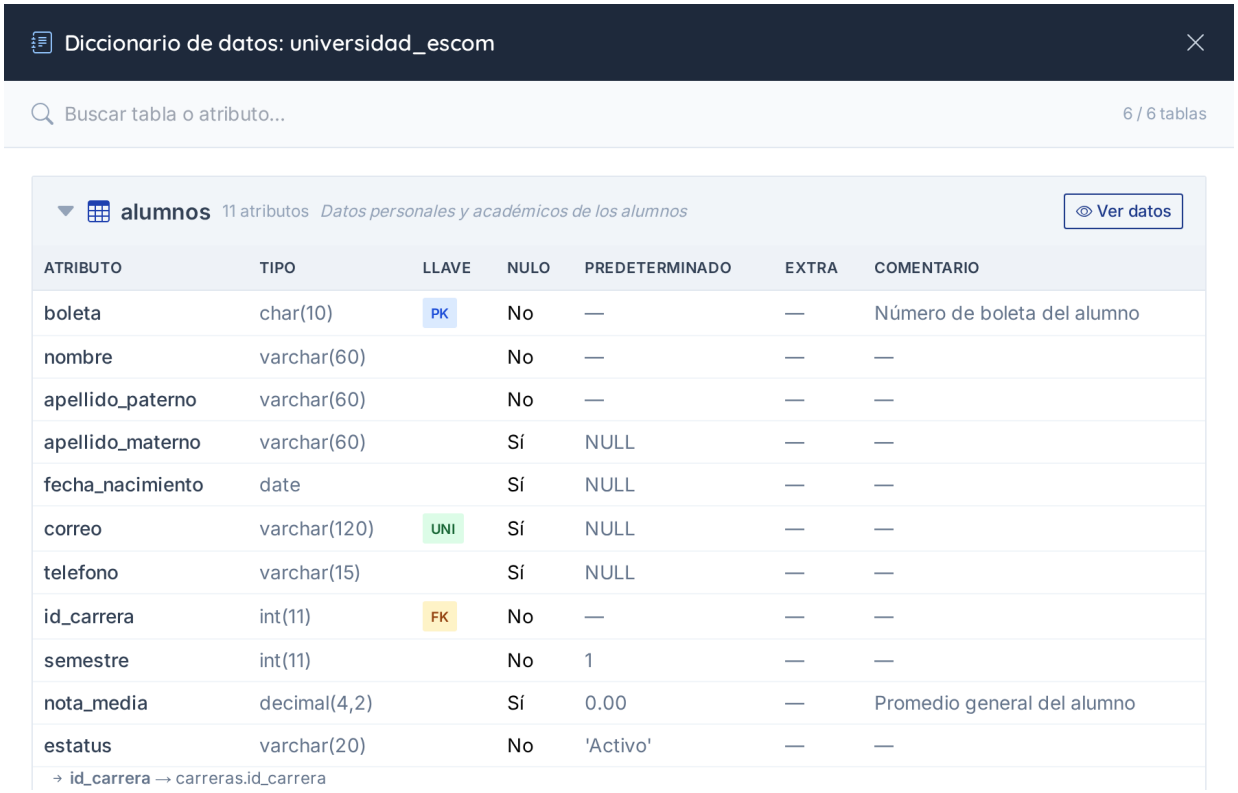


Figura 11. Diccionario de datos con la tabla alumnos expandida.

Para cada tabla se muestra:

- Nombre y comentario de la tabla.
- Número de atributos.
- Lista de columnas con:
  - Nombre.
  - Tipo (`varchar(60)`, `decimal(4,2)`, `date`, etc.).
  - Llave: `PK` (primaria), `UNI` (única) o `FK` (foránea).
  - Si admite nulos.
  - Valor predeterminado.
  - Atributos extra (por ejemplo, `auto_increment`).
  - Comentario de la columna.
- Relaciones de clave foránea, con el formato `columna → tabla.columna`.
- Botón **Ver datos**.

Haz clic en el encabezado de una tabla para expandir o contraer sus columnas. El buscador filtra por nombre de tabla **o** de atributo.

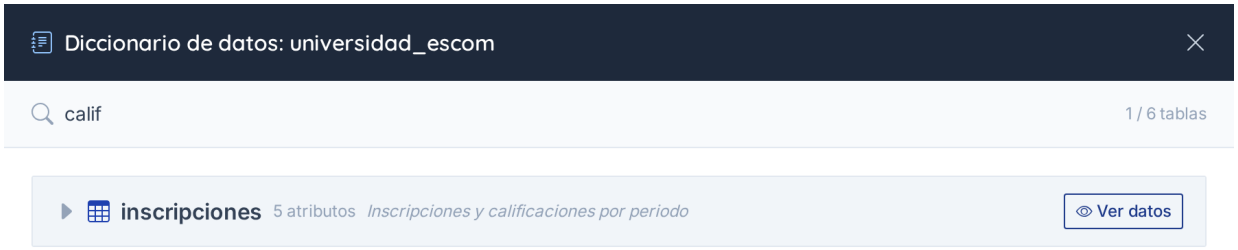


Figura 12. Búsqueda de tablas y atributos en el diccionario.

El contador de la derecha indica cuántas tablas coinciden con la búsqueda respecto del total.

3.8. Vista previa del contenido de una tabla

Haz clic en el nombre de una tabla o en **Ver datos** para abrir una ventana modal con las primeras 50 filas.

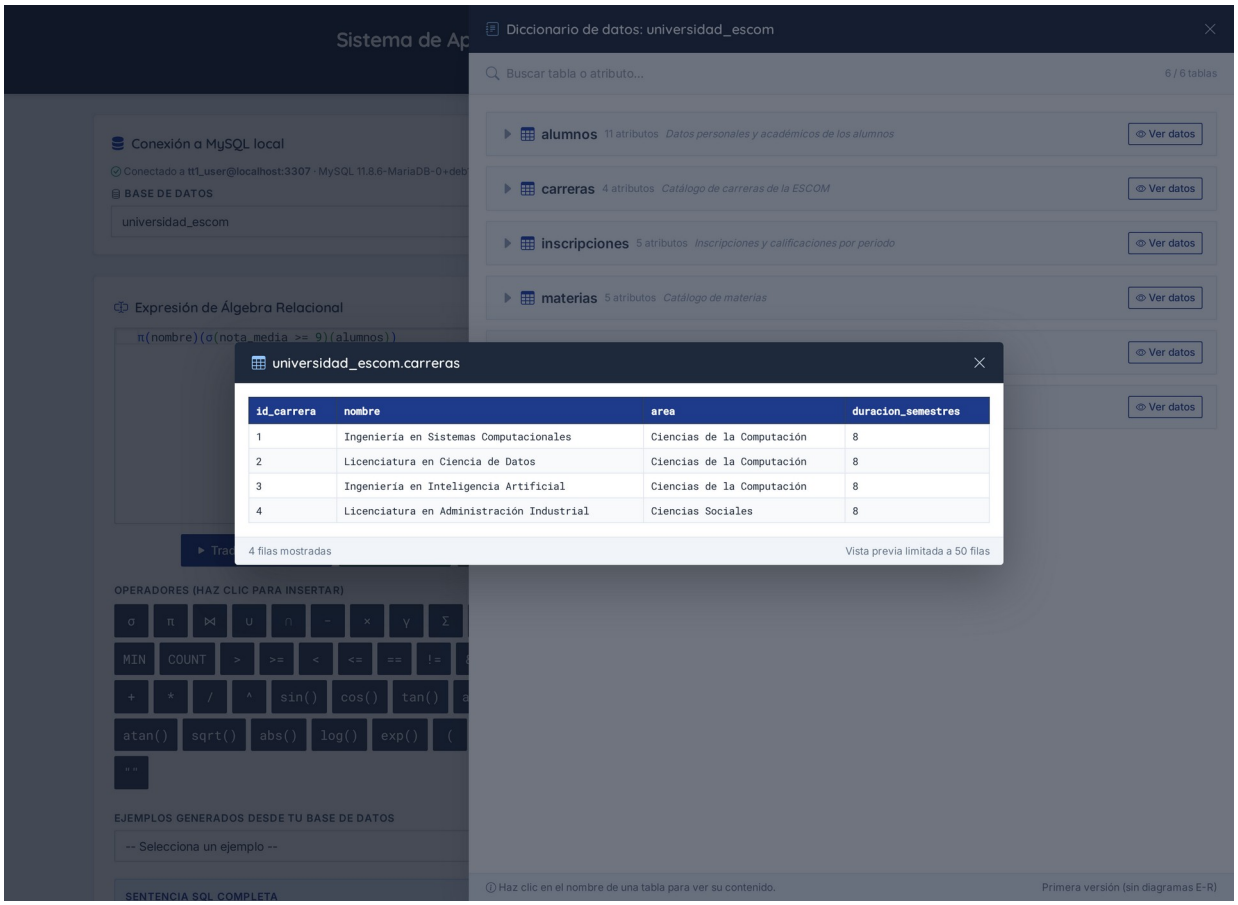


Figura 13. Vista previa de la tabla carreras.

La vista previa es de solo lectura y está limitada a 50 filas para no saturar el navegador. Si la tabla tiene más columnas de las que caben, aparece desplazamiento horizontal. Los valores `NULL` se muestran como `NULL` en cursiva.

3.9. Historial de variables

El historial aparece en la parte inferior del editor como **Historial activo**. Al principio está vacío.

Cuando guardas una relación con **Guardar @n**, el sistema:

1. Pide un nombre con formato `@` seguido de números (por ejemplo, `@1`).
2. Antepone `nombre <-` a la última expresión traducida.
3. La traduce y guarda el resultado en memoria.
4. Muestra el nombre en el historial.



Figura 14. Guardado de una relación en la variable @1.

Después puedes usar `@1` en una nueva expresión:

```
π(nombre, nota_media)(@1)
```



Figura 15. Reutilización de la variable @1 en una nueva consulta.

Las variables viven en la memoria del servidor. Si reinicias la aplicación, se pierden. **Limpiar historial** las elimina todas, previa confirmación.

3.10. Barra de tip y página Acerca de

La primera vez que abres la aplicación aparece una barra inferior con un tip: abrir el diccionario y hacer clic en una tabla para ver sus datos. Puedes cerrarla con la **X**; la decisión se guarda en `sessionStorage`, así que la barra no vuelve a aparecer en esa pestaña.

El enlace **Acerca de** abre una página con la descripción del proyecto, sus funcionalidades, la arquitectura y el estado de la primera versión.



Figura 16. Página Acerca de la herramienta.

4. Fundamentos y sintaxis del álgebra relacional

4.1. Elementos léxicos

El analizador léxico reconoce los siguientes elementos:

Elemento	Ejemplos	Notas
Operadores relacionales	σ, π, ⋈, ∪, ∩, γ, Σ	Símbolos Unicode.
Palabras de operadores	`SELEC`, `PROY`, `JOIN`, `UNION`, `INTERSEC`, `DIF`, `PRODCART`, `AGRUPAR`	No distinguen mayúsculas de minúsculas.
Funciones de agregación	`SUM`, `PROM`, `AVG`, `MAX`, `MIN`, `COUNT`	También `SUMA`, `PROMEDIO`, `CONTAR`.
Funciones matemáticas	`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`, `sqrt`, `raiz`, `log`, `ln`, `exp`, `abs`, `floor`, `ceil`, `round`, `mod`, `pow`, `radians`, `degrees`	Se traducen a funciones SQL.
Comparadores	>, >=, <, <=, ==, !=, <>	== y != son los recomendados en la gramática.
Operadores lógicos	&&, `AND`, ||, `OR`, !, `NOT`	También se aceptan las palabras.
Aritméticos	+, -, *, /, ^	^ es potencia.
Asignación	<- o =	Solo para variables de historial.
Delimitadores	(,), ,, ;	La coma separa atributos; el punto y coma separa listas en algunas formas.
Números	`18`, `9.5`, `1e3`	Decimales y notación científica.
Cadenas	`"Activo"`, `'Activo'`	Comillas simples o dobles.
Booleanos y nulo	`true`, `false`, `null`	No distinguen mayúsculas.
Variables	`@1`, `@2`, `@10`	Solo dígitos después de @.
Identificadores	`alumnos`, `nota_media`, `id_carrera`	Letras, dígitos y guion bajo; deben empezar con letra o guion bajo.
Identificadores cualificados	`alumnos.nombre`	Formato `tabla.atributo`.
Comentarios	`-- comentario`, `// comentario`, `/* bloque */`	Se ignoran.

4.2. Formas de escribir una operación

La gramática acepta dos estilos:

Forma funcional o prefija

```
SELEC(condición, relación)
SELEC(condición)(relación)
PROY(lista_de_atributos, relación)
PROY(lista_de_atributos; relación)
JOIN(relación1, relación2)
JOIN(relación1, relación2, condición)
UNION(relación1, relación2)
INTERSEC(relación1, relación2)
DIF(relación1, relación2)
PRODCART(relación1, relación2)
AGRUPAR(atributos; agregaciones; relación)
```

Forma simbólica o infija

```
σ(condición)(relación)
π(atributos)(relación)
R1 ⋈ R2
R1 ∪ R2
R1 ∩ R2
R1 − R2
R1 × R2
γ(atributos; agregaciones)(relación)
```

Las dos formas producen el mismo resultado. La forma simbólica es la que se usa normalmente en los libros de texto; la funcional es útil cuando no puedes escribir símbolos Unicode o cuando prefieres una sintaxis más parecida a una función.

> ***No mezcles paréntesis y comas de forma incorrecta.*** *`σ(condición)(relación)` lleva dos pares de paréntesis; `SELEC(condición, relación)` lleva uno solo y una coma. Ambas son válidas, pero `σ(condición, relación)` no lo es.*

4.3. Nombres de relaciones y atributos

- Los nombres de tabla y de columna deben existir en la base de datos seleccionada.
- Los identificadores no distinguen mayúsculas de minúsculas en el analizador, pero MySQL puede comportarse de forma distinta según el sistema operativo y la configuración. Para evitar problemas, usa el mismo nombre que aparece en el diccionario.
- Puedes usar nombres cualificados: `alumnos.nombre`, `carreras.area`. Son útiles en condiciones de reunión.
- No se admiten nombres entre comillas invertidas (`` ` ``) ni alias con `AS`.
- Si un atributo coincide con una palabra reservada, puede interpretarse como operador. Por ejemplo, una columna llamada `promedio` se confunde con la palabra `PROM`; en ese caso puedes escribir `alumnos.promedio` o cambiar el nombre de la columna. Lo mismo ocurre con `suma`, `max`, `min`, `pi`, `sigma`, `union`, etc.

4.4. Literales

Tipo	Ejemplos	Traducción a SQL
Número	`18`, `9.5`, `1e3`	`18`, `9.5`, `1e3`
Cadena	`"Activo"`, `'Ana'`	`'Activo'`, `'Ana'`
Verdadero	`true`	`TRUE`
Falso	`false`	`FALSE`
Nulo	`null`	`NULL`

Las cadenas se pueden comparar con atributos de tipo `VARCHAR`, `CHAR`, `DATE`, etc. MySQL convierte implícitamente en algunos casos, pero es mejor que los tipos coincidan. Por ejemplo, para una fecha:

```
σ(fecha_nacimiento >= "2005-01-01")(alumnos)
```

4.5. Selección (σ)

Sintaxis:

```
σ(condición)(R)
SELEC(condición, R)
SELEC(condición)(R)
```

Significado: devuelve las tuplas de `R` que hacen verdadera la condición. Es equivalente a la cláusula `WHERE` de SQL.

Ejemplo:

```
σ(nota_media >= 9)(alumnos)
```

SQL generado:

```
SELECT * FROM alumnos WHERE (nota_media >= 9)
```

La condición puede combinar comparadores y operadores lógicos:

```
σ(id_carrera == 1 && nota_media >= 8)(alumnos)
```

SQL generado:

```
SELECT * FROM alumnos WHERE ((id_carrera = 1) AND (nota_media >= 8))
```

4.6. Proyección (π)

Sintaxis:

```
π(lista_de_atributos)(R)
PROY(lista_de_atributos, R)
PROY(lista_de_atributos; R)
```

Significado: devuelve solo las columnas indicadas. Es equivalente a la lista de columnas del `SELECT` en SQL.

Ejemplo:

```
π(nombre, nota_media)(alumnos)
```

SQL generado:

```
SELECT nombre, nota_media FROM alumnos
```

La lista de proyección puede incluir expresiones, no solo nombres de columnas:

```
π(nombre, nota_media * 1.1)(alumnos)
```

SQL generado:

```
SELECT nombre, (nota_media * 1.1) FROM alumnos
```

> ***Diferencia teórica importante.*** *En álgebra relacional, la proyección elimina duplicados porque el resultado es un conjunto. El sistema genera un `SELECT` sin `DISTINCT`, así que SQL puede devolver filas repetidas si la tabla las tiene. Si necesitas el comportamiento conjuntista, tendrías que agregar `DISTINCT` en SQL; la interfaz no lo hace por ti.*

4.7. Reunión (⋈)

La reunión combina tuplas de dos relaciones que cumplen una condición de coincidencia. El sistema soporta dos variantes:

Reunión natural

```
R1 ⋈ R2
JOIN(R1, R2)
```

Se traduce a `NATURAL JOIN`. SQL busca **todos los nombres de columna que aparezcan en ambas relaciones** y une por igualdad en todos ellos.

Ejemplo:

```
alumnos ⋈ π(id_carrera, area)(carreras)
```

SQL generado:

```
SELECT * FROM alumnos
NATURAL JOIN (SELECT id_carrera, area FROM carreras) AS _q...
```

La proyección sobre `carreras` es intencional: si se unen `alumnos` y `carreras` completas, ambas tienen una columna `nombre`, y la reunión natural intentaría igualar también los nombres de alumno y de carrera, lo que normalmente produce cero filas. Al proyectar solo `id_carrera` y `area`, la única columna común es `id_carrera`.

Reunión con condición (theta join)

```
JOIN(R1, R2, condición)
```

Se traduce a `JOIN ... ON condición`.

Ejemplo:

```
JOIN(alumnos, carreras, alumnos.id_carrera == carreras.id_carrera)
```

SQL generado:

```
SELECT * FROM alumnos
JOIN carreras ON (alumnos.id_carrera = carreras.id_carrera)
```

> ***Cuidado con las columnas repetidas.*** *Al hacer `SELECT *` de una reunión, las columnas con el mismo nombre pueden aparecer dos veces. La tabla de resultados del sistema identifica las columnas por su nombre, así que una puede sobrescribir a la otra. Para evitar confusiones, proyecta columnas con nombres distintos o usa una reunión natural con una proyección que elimine la ambigüedad.*

4.8. Operaciones de conjuntos

Unión (∪)

```
R1 ∪ R2
UNION(R1, R2)
```

Devuelve las tuplas que están en `R1`, en `R2` o en ambas. SQL usa `UNION`, que elimina duplicados por defecto.

Ejemplo:

```
π(nombre)(alumnos) ∪ π(nombre)(profesores)
```

SQL generado:

```
SELECT nombre FROM alumnos
UNION
SELECT nombre FROM profesores
```

Intersección (∩)

```
R1 ∩ R2
INTERSEC(R1, R2)
```

Devuelve las tuplas que están en ambas relaciones. SQL usa `INTERSECT`.

Ejemplo:

```
π(nombre)(alumnos) ∩ π(nombre)(profesores)
```

Diferencia (–)

```
R1 – R2
R1 - R2
DIF(R1, R2)
```

Devuelve las tuplas de `R1` que no están en `R2`. SQL usa `EXCEPT` (en MySQL 8.0.31+ y MariaDB; en versiones anteriores puede no estar disponible).

Ejemplo:

```
π(nombre)(alumnos) – π(nombre)(profesores)
```

> ***Compatibilidad de conjuntos.*** *Para que una unión, intersección o diferencia funcione, las dos relaciones deben tener el mismo número de columnas y tipos compatibles. El sistema no valida esto antes de enviar la consulta; MySQL devolverá un error si no se cumple.*

4.9. Producto cartesiano (×)

```
R1 × R2
R1 * R2
PRODCART(R1, R2)
```

Combina cada tupla de `R1` con cada tupla de `R2`. Si `R1` tiene `m` filas y `R2` tiene `n`, el resultado tiene `m × n` filas.

Ejemplo:

```
π(nombre)(alumnos) × π(id_carrera)(carreras)
```

SQL generado:

```
SELECT * FROM (SELECT nombre FROM alumnos) AS _q1
CROSS JOIN (SELECT id_carrera FROM carreras) AS _q2
```

El producto cartesiano rara vez se usa solo; normalmente se combina con una selección que impone la condición de reunión.

4.10. Agrupación y agregación (γ)

Sintaxis:

```
γ(atributos_de_agrupación; funciones_de_agregación)(R)
AGRUPAR(atributos; agregaciones; R)
GROUP(atributos; agregaciones)(R)
```

Significado: agrupa las tuplas por los atributos indicados y calcula una o más funciones de agregación. Es equivalente a `GROUP BY` en SQL.

Ejemplo:

```
γ(id_carrera; COUNT(*), AVG(nota_media))(alumnos)
```

SQL generado:

```sql
SELECT id_carrera, COUNT(*), AVG(nota_media)
FROM alumnos
GROUP BY id_carrera
```

Funciones disponibles:

Función	Palabras aceptadas	Significado	Ejemplo
SUM	SUM, SUMA, Σ	Suma	SUM(creditos)
AVG	AVG, PROM, PROMEDIO	Promedio	AVG(nota_media)
MAX	MAX	Máximo	MAX(nota_media)
MIN	MIN	Mínimo	MIN(nota_media)
COUNT	COUNT, CONTAR	Conteo	COUNT(*), COUNT(boleta)

`COUNT(*)` cuenta tuplas; `COUNT(atributo)` cuenta valores no nulos de ese atributo.

> ***Orden de los argumentos.*** *En γ, primero van los atributos de agrupación, luego un punto y coma, después las funciones de agregación y finalmente la relación. No olvides el punto y coma:* `γ(id_carrera; COUNT(*))(alumnos)` *es correcto;* `γ(id_carrera, COUNT(*))(alumnos)` *no.*

4.11. Expresiones aritméticas y comparaciones

Dentro de una condición o de una lista de proyección puedes usar expresiones escalares:

```
nota_media * 1.1
(nota_media + 2) / 3
nota_media ^ 2
- nota_media
abs(nota_media - 9)
```

Comparadores:

Operador AR	SQL	Significado
==	=	Igual a
!= o <>	<>	Distinto de

Operador AR	SQL	Significado
>	>	Mayor que
>=	>=	Mayor o igual que
<	<	Menor que
<=	<=	Menor o igual que

> ***No uses = para comparar.*** *En la gramática, = se interpreta como asignación (`@1 = ...`). Para comparar igualdad usa ==. El sistema traduce == a = en SQL.*

4.12. Operadores lógicos

Operador	Alternativas	Significado
&&	AND, and	Conjunción: ambas condiciones deben cumplirse.
||	OR, or	Disyunción: al menos una condición debe cumplirse.
!	NOT, not	Negación.

Ejemplo:

```
σ(id_carrera == 1 && nota_media >= 8)(alumnos)
```

SQL generado:

```sql
SELECT * FROM alumnos
WHERE ((id_carrera = 1) AND (nota_media >= 8))
```

Puedes agrupar condiciones con paréntesis:

```
σ((id_carrera == 1 || id_carrera == 2) && nota_media > 8)(alumnos)
```

4.13. Funciones matemáticas y trigonométricas

El sistema acepta las siguientes funciones dentro de expresiones escalares:

Función	Significado
`sin(x)`, `cos(x)`, `tan(x)`	Seno, coseno y tangente (radianes).
`asin(x)`, `acos(x)`, `atan(x)`	Arco seno, arco coseno y arco tangente.
`atan2(y, x)`	Arco tangente de dos argumentos.
`sqrt(x)`, `raiz(x)`	Raíz cuadrada.
`log(x)`, `ln(x)`	Logaritmo.
`exp(x)`	Exponencial.
`abs(x)`	Valor absoluto.
`floor(x)`, `ceil(x)`, `round(x)`	Redondeo hacia abajo, hacia arriba y al más cercano.
`mod(x, y)`	Residuo.
`pow(x, y)`	Potencia.
`radians(x)`, `degrees(x)`	Conversión entre grados y radianes.

Ejemplo:

```
π(nombre, sin(nota_media))(alumnos)
```

SQL generado:

```
SELECT nombre, SIN(nota_media) FROM alumnos
```

Las funciones se pueden usar en condiciones:

```
π(nombre)(σ(abs(nota_media - 9) < 0.5)(alumnos))
```

4.14. Variables de historial y asignación

```
@1 <- σ(nota_media >= 8)(alumnos)
```

Guarda la relación resultante en `@1`. Después puedes usarla:

```
π(nombre, nota_media)(@1)
```

La asignación también acepta `=`:

```
@1 = σ(nota_media >= 8)(alumnos)
```

Pero para evitar ambigüedad con la igualdad, se recomienda `<-`.

Reglas:

- El nombre debe ser `@` seguido de uno o más dígitos.
- La asignación se guarda en la memoria del servidor.
- Las variables se sustituyen antes de generar el SQL final.
- Si usas una variable que no existe, el sistema genera SQL con `@1` literal y MySQL devuelve un error de sintaxis. Guarda la variable primero.

4.15. Comentarios, espacios y mayúsculas

- Los comentarios de línea empiezan con `--` o `//` y llegan hasta el final de la línea.
- Los comentarios de bloque usan `/* ... */` y pueden ocupar varias líneas.
- Los espacios, tabuladores y saltos de línea se ignoran.
- Las palabras clave no distinguen mayúsculas de minúsculas: `SELEC`, `selec` y `SeLeC` son equivalentes.
- Los nombres de tablas y columnas se pasan a MySQL tal como se escriben; procura usar el mismo caso que en el diccionario.

4.16. Precedencia de operadores

De menor a mayor prioridad:

1. `OR` / `||`
2. `AND` / `&&`
3. `NOT` / `!`
4. Comparadores (`==`, `!=`, `>`, `>=`, `<`, `<=`)
5. `UNION`, `INTERSEC`, `DIFFERENCE`
6. `JOIN`
7. `+`, `-`

8. `PRODUCT`, `*`, `/`
9. `^` (potencia, asociativa por la derecha)
10. `-` unario

Cuando tengas duda, usa paréntesis. Son gratuitos y hacen la expresión más legible.

4.17. Palabras reservadas y colisiones

Las siguientes palabras se interpretan como operadores, funciones o valores, no como nombres de columna:

```
selec, seleccion, selección, sigma
proy, proyeccion, proyección, pi
join, union, inter, intersec, interseccion, intersección
dif, diferencia, prodcart, producto, agrupar, agrup
sum, suma, prom, promedio, avg, max, min, count, contar
sin, cos, tan, asin, acos, atan, atan2, sqrt, raiz, log, ln, exp,
abs, floor, ceil, round, mod, pow, radians, degrees
and, or, not, true, false, null
```

Si una columna se llama igual que una de estas palabras, el analizador puede confundirla. Ejemplos típicos: `promedio`, `suma`, `max`, `min`, `pi`, `sigma`. Soluciones:

- Renombrar la columna (por ejemplo, `nota_media` en lugar de `promedio`).
- Escribir el nombre cualificado (`alumnos.promedio`), que el lexer reconoce como un solo token `QUALIFIED`.
- Evitar esos nombres al crear tablas de práctica.

4.18. Reglas semánticas y operaciones no incluidas

El analizador verifica la **forma** de la expresión, no su **semántica**. Que una expresión sea sintácticamente válida no garantiza que las tablas existan, que las columnas pertenezcan a ellas o que los tipos sean compatibles. Esos errores aparecen al ejecutar en MySQL.

Operaciones que **no** están implementadas en esta versión:

- Renombramiento (ρ).
- División relacional.
- Reunión externa (`LEFT JOIN`, `RIGHT JOIN`, `FULL JOIN`).
- Subconsultas.
- `ORDER BY`, `LIMIT`, `HAVING`.
- `DISTINCT` explícito.
- Álgebra relacional con bolsas (multiconjuntos) declarada de forma explícita.
- Vistas y tablas temporales.
- Funciones de fecha, cadena o conversión distintas de las listadas.
- Alias (`AS`) en álgebra relacional.

5. Tutoriales guiados

Los tutoriales usan la base `universidad_escom`. Si aún no la cargaste, revisa la sección 2.6. Las capturas muestran un servidor en el puerto `3307`; en tu equipo usa el puerto que corresponda.

5.1. Tutorial 1: conectar y explorar la base de datos

Objetivo: conectarse, seleccionar la base y revisar el diccionario.

1. Abre `http://localhost:8080`.
2. Escribe los datos de conexión y pulsa **Conectar**.
3. Verifica que aparezca **Conectado** y que el selector muestre `universidad_escom`.
4. Pulsa **Ver diccionario de datos**.
5. Expande la tabla `alumnos`.
6. Revisa las columnas, tipos, claves y el comentario de la tabla.
7. Pulsa **Ver datos** en `alumnos` para confirmar que hay 10 filas.
8. Cierra la vista previa y el diccionario.

Resultado esperado: puedes ver la estructura y los datos reales de la base. Si el diccionario aparece vacío, revisa que hayas seleccionado una base con tablas y que el usuario tenga permisos.

5.2. Tutorial 2: primera consulta

Objetivo: traducir una selección y ejecutarla.

1. En el editor escribe:

```
σ(nota_media >= 9)(alumnos)
```

2. Pulsa **Traducir y ejecutar**.
3. Revisa los tokens: `σ`, `(`, `nota_media`, `>=`, `9`, `)`, `(`, `alumnos`, `)`.
4. Revisa el SQL generado:

```
SELECT * FROM alumnos WHERE (nota_media >= 9)
```

5. Revisa el árbol: la raíz es **Selección (σ)**, con una condición y una fuente.
6. Revisa la tabla de resultados: deben aparecer Ana María, Valeria y Jorge (3 filas).

Explicación: σ filtra filas; `nota_media >= 9` es la condición; `alumnos` es la relación de entrada.

5.3. Tutorial 3: selección y proyección anidadas

Objetivo: combinar operadores y entender el orden de evaluación.

1. Escribe:

```
π(nombre, nota_media)(σ(id_carrera == 1 && nota_media >= 8)(alumnos))
```

2. Traduce y ejecuta.

3. Observa que el SQL usa una subconsulta:

```
SELECT nombre, nota_media
FROM (SELECT * FROM alumnos
      WHERE ((id_carrera = 1) AND (nota_media >= 8))) AS _q...
```

4. Revisa el árbol: la raíz es **Proyección (π)** y su fuente es una **Selección (σ)**.
5. El resultado debe tener 3 filas: Ana María, Emiliano y Jorge.

Explicación: primero se filtran las filas de la carrera 1 con nota media de 8 o más; después se conservan solo `nombre` y `nota_media`.



Figura 17. Traducción de una proyección sobre una selección anidada.

5.4. Tutorial 4: reunión de dos tablas

Objetivo: combinar alumnos con carreras y evitar la ambigüedad de `nombre`.

1. Escribe:

```
π(nombre, area)(alumnos ⋈ π(id_carrera, area)(carreras))
```

2. Traduce y ejecuta.
3. Revisa el SQL: se usa `NATURAL JOIN` y una subconsulta para `carreras`.
4. Revisa el árbol: **Proyección** → **Reunión natural** → `alumnos` y **Proyección** sobre `carreras`.
5. El resultado debe tener 10 filas con `nombre` y `area`.

Explicación: `alumnos` y `carreras` comparten `id_carrera`. La proyección sobre `carreras` deja solo `id_carrera` y `area`, de modo que la reunión natural no intente igualar también `nombre`.

> ***Prueba este error didáctico.*** *Ejecuta `alumnos ⋈ carreras` sin la proyección. La reunión natural iguala `id_carrera` y `nombre`, por lo que probablemente obtendrás 0 filas. Es un excelente ejemplo de por qué hay que conocer el esquema.*



Figura 18. Traducción de una reunión natural con proyección.

5.5. Tutorial 5: agrupación y agregación

Objetivo: calcular el número de alumnos y el promedio por carrera.

1. Escribe:

```
γ(id_carrera; COUNT(*), AVG(nota_media))(alumnos)
```

2. Traduce y ejecuta.
3. Revisa el SQL:

```
SELECT id_carrera, COUNT(*), AVG(nota_media)
FROM alumnos
GROUP BY id_carrera
```

4. Revisa el árbol: **Agrupación (γ)** con atributos, funciones de agregación y fuente.
5. El resultado debe tener 3 filas (carreras 1, 2 y 3).

Explicación: `id_carrera` es el atributo de agrupación; `COUNT(*)` cuenta alumnos por grupo; `AVG(nota_media)` calcula el promedio por grupo.

Proceso de traducción

TOKENIZACIÓN

γ	(	id_carrera	;	COUNT	(	*
GROUP	LPAREN	IDENTIFIER	SEMICOLON	COUNT	LPAREN	TIMES

)	,	AVG	(	nota_media	)	)	(
RPAREN	COMMA	AVG	LPAREN	IDENTIFIER	RPAREN	RPAREN	LPAREN

alumnos	)
IDENTIFIER	RPAREN

EQUIVALENCIA AR → SQL

ÁLGEBRA RELACIONAL

```
γ(id_carrera; COUNT(*),
AVG(nota_media))(alumnos)
```

→

SQL GENERADO

```
SELECT id_carrera, COUNT(*),
AVG(nota_media) FROM alumnos GROUP
BY id_carrera
```

ÁRBOL SINTÁCTICO

```
Agrupacion (γ)
  Lista de atributos/expresiones:
    Atributo: id_carrera
  Funciones de agregacion:
    Agregacion: COUNT()
      *
    Agregacion: AVG()
      Atributo: nota_media
  Fuente:
    Tabla: alumnos
```

TABLA DE RESULTADOS — 3 filas, 4 ms

id_carrera	COUNT(*)	AVG(nota_media)
1	6	7.808333
2	2	8.1
3	2	9.15

Figura 19. Traducción de una agrupación con dos funciones de agregación.

5.6. Tutorial 6: funciones matemáticas

Objetivo: usar una función dentro de una proyección.

1. Escribe:

```
π(nombre, sin(nota_media))(alumnos)
```

2. Traduce y ejecuta.
3. Revisa el SQL:

```
SELECT nombre, SIN(nota_media) FROM alumnos
```

4. Observa que la columna calculada se llama `SIN(nota_media)`.
5. Prueba también:

```
π(nombre)(σ(abs(nota_media - 9) < 0.5)(alumnos))
```

Explicación: las funciones se evalúan por fila. `sin` recibe el valor de `nota_media` en radianes; `abs` devuelve el valor absoluto.

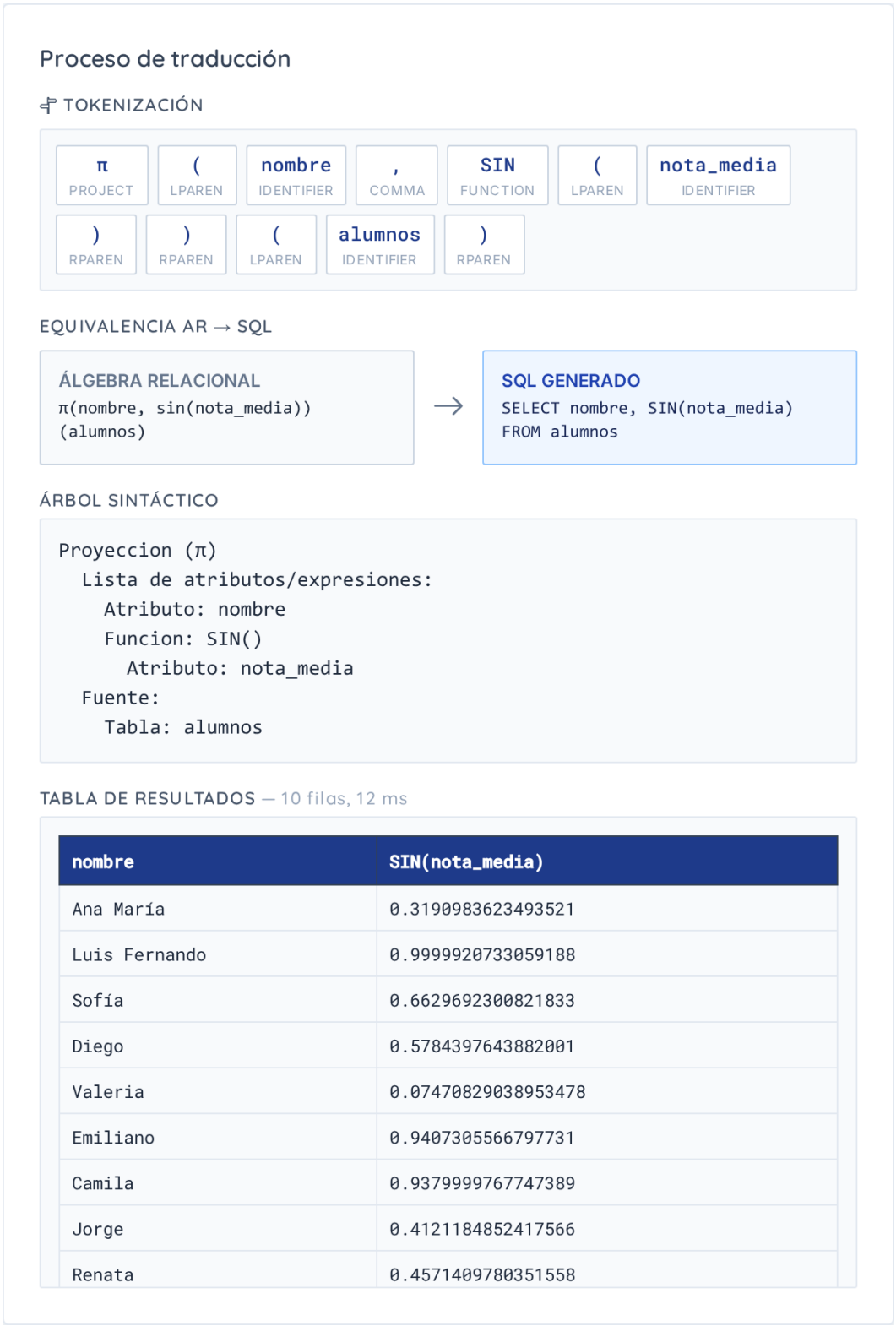


Figura 20. Traducción de una función trigonométrica dentro de una proyección.

5.7. Tutorial 7: guardar y reutilizar una relación

Objetivo: usar el historial para no repetir una consulta.

1. Escribe y traduce:

```
σ(nota_media >= 8)(alumnos)
```

2. Pulsa **Guardar @n**.
3. Acepta el nombre sugerido o escribe `@1`.
4. Observa que el editor ahora contiene `@1 <- σ(nota_media >= 8)(alumnos)` y que el historial muestra `@1`.
5. Escribe:

```
π(nombre, nota_media)(@1)
```

6. Traduce y ejecuta. El SQL debe incluir la selección guardada como subconsulta y devolver 6 filas.
7. Prueba **Limpiar historial** y verifica que `@1` desaparezca.

Explicación: una variable guarda una relación completa, no una cadena de texto. Al usarla, el sistema la expande y genera el SQL equivalente.

5.8. Tutorial 8: provocar y corregir errores

Objetivo: aprender a leer los mensajes del sistema.

Error sintáctico

Escribe:

```
σ(nota_media >= 9)(alumnos
```

Falta el paréntesis de cierre. El sistema muestra `[SIN-01]`, el token encontrado (`fin de la expresión`), la línea/columna y una lista de tokens esperados. Corrige agregando `)`.

Error léxico

Escribe:

```
σ(nota_media >= 9)(alumnos) ?
```

El carácter `?` no pertenece al alfabeto. El sistema muestra `[LEX-01]` con la línea y la columna del carácter. Elimínalo.

Error de ejecución

Escribe:

```
π(nombre)(alumnos_que_no_existen)
```

La expresión es sintácticamente válida, pero la tabla no existe. El SQL se genera y MySQL devuelve un error en la tabla de resultados. Corrige el nombre de la tabla usando el diccionario.

5.9. Tutorial 9: una sesión completa

Este es un flujo típico de clase:

1. Conectarse a MySQL.
2. Abrir el diccionario y revisar `alumnos`, `carreras` e `inscripciones`.
3. Ejecutar `σ(nota_media >= 9)(alumnos)`.
4. Ejecutar `π(nombre, nota_media)(σ(id_carrera == 1 && nota_media >= 8)(alumnos))`.
5. Ejecutar `π(nombre, area)(alumnos ⋈ π(id_carrera, area)(carreras))`.
6. Ejecutar `γ(id_carrera; COUNT(*), AVG(nota_media))(alumnos)`.
7. Guardar la selección de aprobados en `@1` y consultarla.
8. Provocar un error y corregirlo.
9. Cerrar la conexión con **Desconectar**.

6. Errores y diagnóstico

6.1. Cómo se presentan los errores

Cuando la expresión no puede procesarse, aparece un recuadro rojo en la parte superior derecha con:

- El código del error entre corchetes, por ejemplo `[SIN-01]`.
- Un mensaje principal.
- Una descripción del catálogo.
- La línea y la columna, si están disponibles.
- La lista de tokens que el analizador esperaba, cuando aplica.
- La tokenización parcial, para que veas hasta dónde se reconoció la expresión.

Los errores de ejecución de MySQL no usan el catálogo; se muestran en la tabla de resultados con el mensaje devuelto por el servidor.

6.2. Catálogo de errores

Código	Nombre	Significado	Ejemplo	Solución
`LEX-01`	Carácter no reconocido	El símbolo no pertenece al alfabeto del álgebra relacional.	`?`, `#`, `&` solo, comillas sin cerrar.	Elimina o corrige el carácter.
`LEX-99`	Error interno del analizador léxico	Fallo inesperado al leer la entrada.	Entrada con codificación extraña.	Revisa el texto y vuelve a intentar.
`SIN-00`	Expresión vacía	No se ingresó ninguna expresión.	Cadena vacía o solo espacios.	Escribe una expresión.
`SIN-01`	Violación sintáctica	La secuencia de tokens no respeta la gramática.	Falta un paréntesis, una coma o una relación.	Revisa la lista de tokens esperados y la línea/columna.
`SIN-02`	Error durante la construcción del árbol	El parser reconoció tokens pero no pudo construir el AST.	Casos límite de la gramática.	Simplifica la expresión y revisa la sintaxis.
`SIS-01`	Error del sistema	Excepción no controlada en el controlador.	Fallo de conexión, error interno.	Revisa la consola del servidor; vuelve a intentar.

6.3. Errores sintácticos frecuentes

Falta un paréntesis

```
σ(nota_media >= 9)(alumnos
```

Mensaje: `[SIN-01] ... se encontró fin de la expresión...` **Solución:** agrega `)`.

Falta una coma

```
π(nombre nota_media)(alumnos)
```

Mensaje: `[SIN-01] ... se encontró IDENTIFIER 'nota_media'...` **Solución:** `π(nombre, nota_media)(alumnos)`.

Se usó una coma en lugar de punto y coma

```
γ(id_carrera, COUNT(*))(alumnos)
```

Solución: `γ(id_carrera; COUNT(*))(alumnos)`.

Se usó = en una comparación

```
σ(nota_media = 9)(alumnos)
```

Mensaje: error sintáctico porque = se interpreta como asignación. **Solución:** `σ(nota_media == 9)(alumnos)`.

Se usó una palabra reservada como columna

```
π(promedio)(alumnos)
```

Mensaje: el lexer reconoce `promedio` como AVG. **Solución:** usa `alumnos.promedio` o renombra la columna.

Paréntesis desbalanceados en una expresión anidada

```
π(nombre)(σ(nota_media >= 9)(alumnos)
```

Solución: cuenta los pares: `π( … )( … ))`. Puedes usar el resaltado de paréntesis de Monaco.

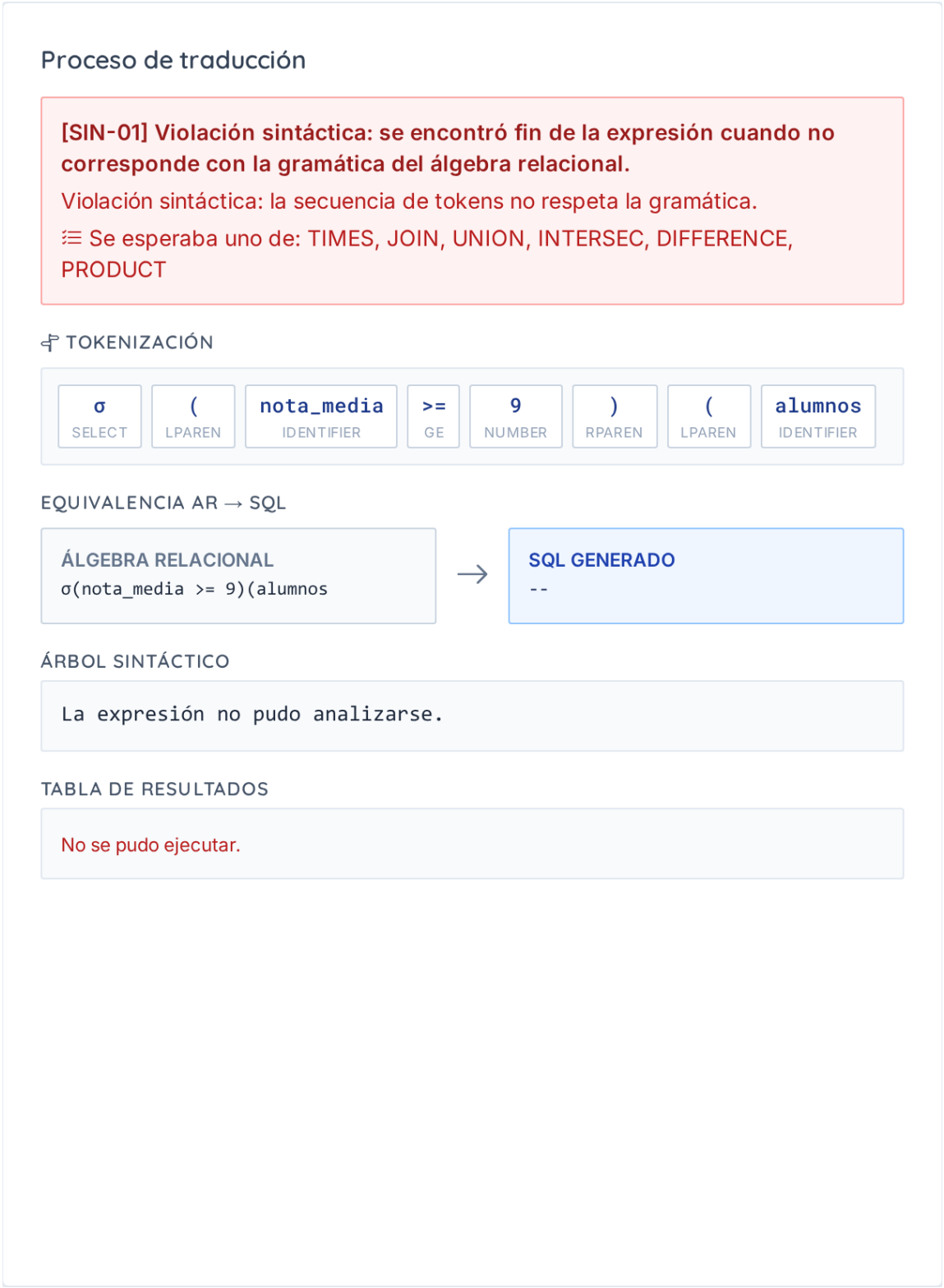


Figura 21. Error sintáctico SIN-01 por falta de un paréntesis de cierre.

Carácter no reconocido

```
σ(nota_media >= 9)(alumnos) ?
```

El mensaje `[LEX-01]` indica el carácter, la línea y la columna. Elimina el carácter o sustitúyelo por uno válido.

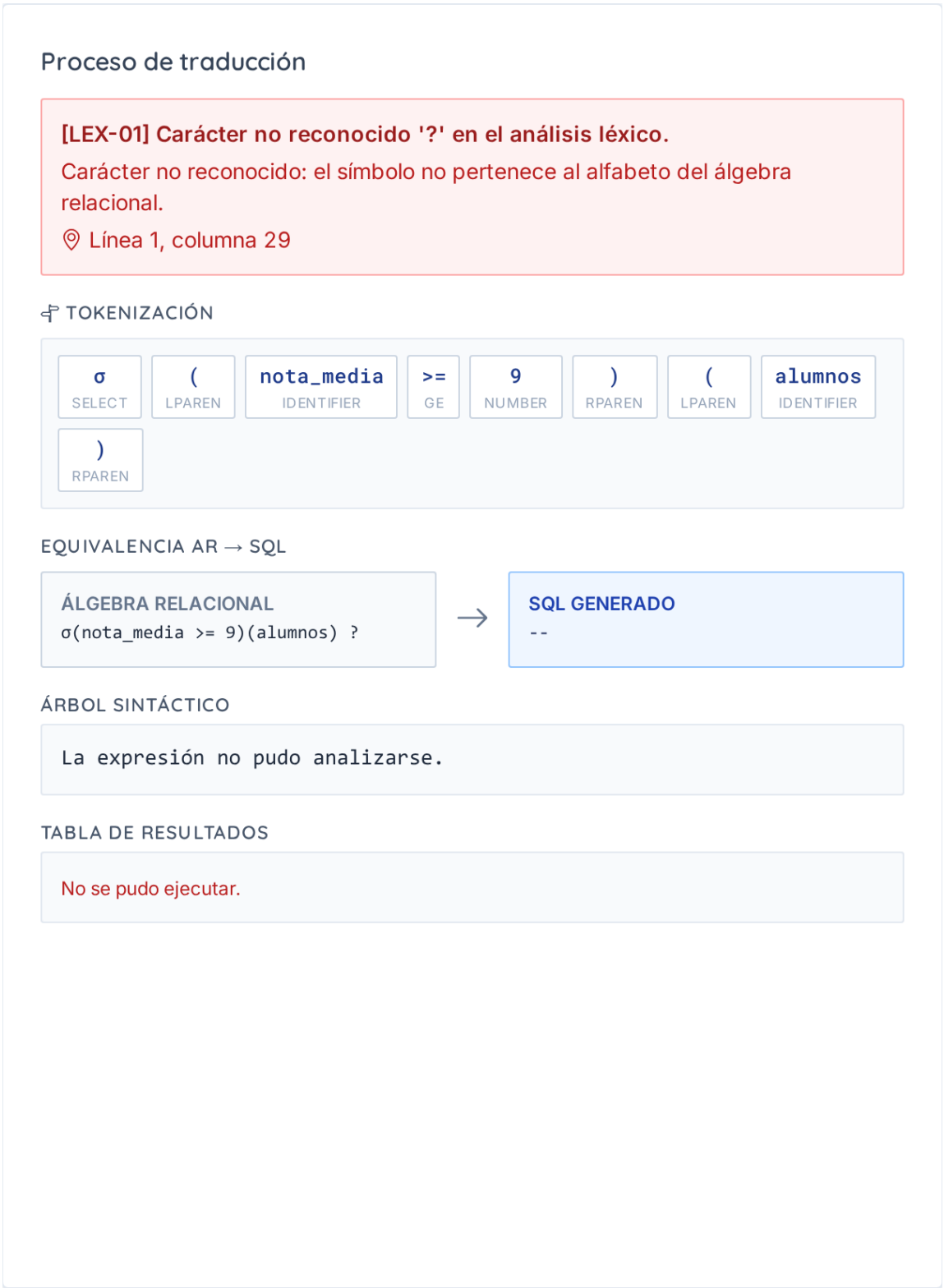


Figura 22. Error léxico LEX-01 por un carácter que no pertenece al alfabeto.

6.4. Errores de ejecución frecuentes

Mensaje de MySQL	Causa	Solución
`Table 'basededatos.tabla' doesn't exist`	La tabla no existe o no está en la base seleccionada.	Revisa el diccionario y el nombre.
`Unknown column 'x' in 'field list'`	La columna no existe.	Revisa el nombre en el diccionario.
`Duplicate column name 'id'`	Una reunión produjo columnas con el mismo nombre dentro de una subconsulta.	Proyecta columnas con nombres distintos o usa una reunión natural con proyección.
`Unknown database`	La base seleccionada ya no existe.	Vuelve a conectarte y selecciona otra.
`Access denied`	El usuario perdió permisos o la conexión expiró.	Vuelve a conectar.
`You have an error in your SQL syntax ... near '@1'`	Usaste una variable que no está guardada.	Guarda la variable primero.
`Operand should contain 1 column`	Una operación de conjuntos tiene distinto número de columnas.	Iguala las listas de proyección.
`Illegal mix of collations`	Las columnas comparadas tienen colaciones distintas.	Convierte o usa columnas compatibles.

6.5. Método de diagnóstico

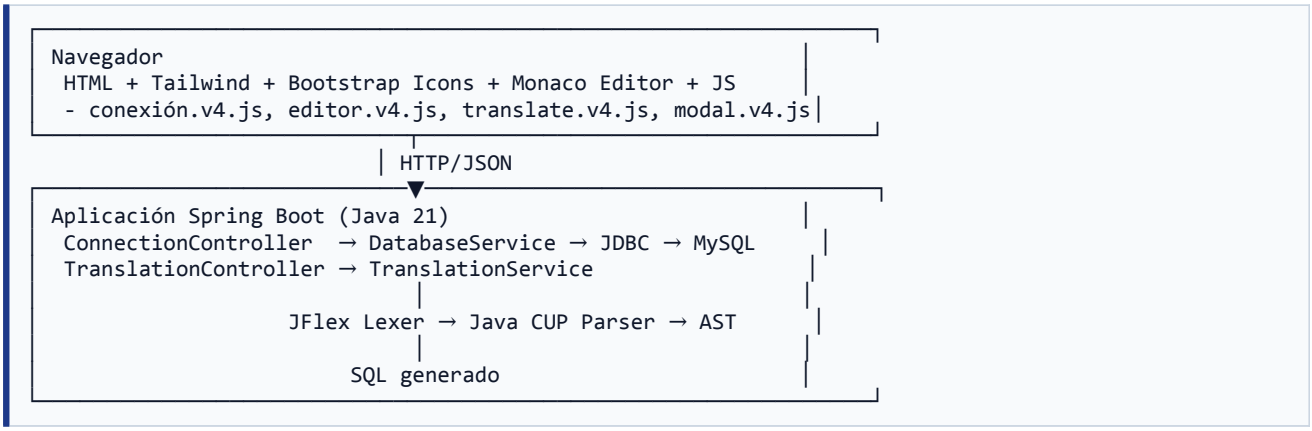
Cuando algo falle, sigue este orden:

1. **Lee el código y la línea/columna.** El sistema indica dónde encontró el problema, no necesariamente dónde está la causa.
2. **Revisa la tokenización.** Si un identificador aparece como un tipo inesperado, hay una colisión de palabras reservadas o un error de escritura.
3. **Cuenta paréntesis y comas.** La mayoría de los errores sintácticos son de delimitadores.
4. **Simplifica.** Prueba primero la tabla sola, luego la selección, luego la proyección, luego la reunión.
5. **Revisa el diccionario.** Confirma nombres de tablas y columnas, tipos y claves.
6. **Revisa la conexión.** Si el SQL es correcto pero no hay resultados, confirma que estés en la base correcta.
7. **Prueba el SQL directamente en MySQL.** Si el SQL generado falla, el problema es de la consulta o del esquema, no del analizador.

7. Cómo funciona el sistema por dentro

Esta sección es opcional para el usuario final, pero ayuda a entender el proyecto y a extenderlo en el Trabajo Terminal II.

7.1. Arquitectura general



Capas:

- **Presentación:** HTML, CSS y JavaScript. Monaco Editor proporciona resaltado, tooltips y autocompletado.
- **Lógica:** Spring Boot expone una API REST. `TranslationService` orquesta el análisis léxico, sintáctico y la traducción. `DatabaseService` gestiona la conexión y las consultas.
- **Datos:** MySQL local, accedido mediante el controlador JDBC de MySQL.

7.2. Flujo de una consulta

1. El navegador envía `POST /api/translate` con `{ expression, database }`.
2. `TranslationService` tokeniza la expresión con `Lexer`.
3. El `parser` de Java CUP valida la gramática y construye un `Expression` (AST).
4. Si hay una asignación, se guarda en el historial y se usa el cuerpo.
5. Se expanden las variables `@n` con las relaciones guardadas.
6. Cada nodo del AST genera su SQL con `toSQL()`.
7. El controlador recibe el SQL y, si hay conexión, llama a `DatabaseService.execute`.
8. `DatabaseService` abre una conexión JDBC, ejecuta la consulta con límite de filas y tiempo, y devuelve columnas y filas.
9. El navegador renderiza tokens, SQL, árbol, resultados o el error.

7.3. Tokenización: un ejemplo paso a paso

Para `σ(nota_media >= 9)(alumnos)`:

#	Lexema	Tipo interno	Descripción
1	σ	SELECT	Operador de selección.
2	(	LPAREN	Paréntesis que abre.
3	nota_media	IDENTIFIER	Nombre de atributo.

#	Lexema	Tipo interno	Descripción
4	>=	GE	Comparador mayor o igual.
5	9	NUMBER	Literal numérico.
6	)	RPAREN	Paréntesis que cierra.
7	(	LPAREN	Paréntesis que abre.
8	alumnos	IDENTIFIER	Nombre de tabla.
9	)	RPAREN	Paréntesis que cierra.

El lexer no sabe si `alumnos` es una tabla o una columna; eso lo decide el parser según el contexto. En la gramática, un `IDENTIFIER` puede ser una tabla o un atributo.

7.4. Gramática (resumen)

La gramática completa está en `src/main/cup/parser.cup`. Un resumen de las producciones principales:

```
program      ::= relation ";" | relation
relation     ::= SELECT "(" expr ")" "(" relation ")"
              | SELECT "(" expr "," relation ")"
              | PROJECT "(" attrList ")" "(" relation ")"
              | PROJECT "(" attrList ";" relation ")"
              | GROUP "(" attrList ";" aggList ")" "(" relation ")"
              | GROUP "(" attrList ";" aggList ";" relation ")"
              | JOIN "(" relation "," relation ")"
              | JOIN "(" relation "," relation "," expr ")"
              | UNION "(" relation "," relation ")"
              | INTERSEC "(" relation "," relation ")"
              | DIFFERENCE "(" relation "," relation ")"
              | PRODUCT "(" relation "," relation ")"
              | VARIABLE "<-" relation
              | VARIABLE
              | IDENTIFIER
              | "(" relation ")"
              | relation UNION relation
              | relation INTERSEC relation
              | relation DIFFERENCE relation
              | relation MINUS relation
              | relation PRODUCT relation
              | relation TIMES relation
              | relation JOIN relation
```

Las producciones de expresiones escalares incluyen aritmética, comparadores, lógica, funciones, literales, atributos y atributos cualificados.

7.5. Reglas de traducción a SQL

Operación AR	SQL generado	Observaciones
Selección `σ(cond)(R)`	`SELECT * FROM R WHERE cond`	La condición se envuelve en paréntesis.
Proyección `π(attrs)(R)`	`SELECT attrs FROM R`	No agrega `DISTINCT`.
Reunión natural `R1 ⋈ R2`	`SELECT * FROM R1 NATURAL JOIN R2`	Iguala todas las columnas comunes.
Reunión theta `JOIN(R1,R2,cond)`	`SELECT * FROM R1 JOIN R2 ON cond`	`INNER JOIN`.
Unión `R1 ∪ R2`	`R1 UNION R2`	Elimina duplicados.
Intersección `R1 ∩ R2`	`R1 INTERSECT R2`	Requiere compatibilidad.
Diferencia `R1 − R2`	`R1 EXCEPT R2`	MySQL 8.0.31+ / MariaDB.

Operación AR	SQL generado	Observaciones
Producto `R1 × R2`	`SELECT * FROM R1 CROSS JOIN R2`	Puede producir muchas filas.
Agrupación `γ(attrs; aggs)(R)`	`SELECT attrs, aggs FROM R GROUP BY attrs`	Sin `HAVING`.
Asignación `@1 <- R`	Se guarda en memoria; el SQL es el de `R`.	No genera tabla.

Las relaciones compuestas se envuelven en subconsultas con alias `_q...` para poder usarlas en `FROM`.

7.6. API REST

Método	Ruta	Función
POST	`/api/connection/connect`	Conecta con `{ host, port, user, password }`.
GET	`/api/connection`	Consulta el estado de la conexión y las bases disponibles.
POST	`/api/connection/disconnect`	Cierra la conexión lógica.
GET	`/api/databases`	Lista las bases de datos de usuario.
GET	`/api/database/{db}/dictionary`	Devuelve tablas, columnas y claves foráneas.
GET	`/api/database/{db}/tables`	Lista las tablas base.
GET	`/api/database/{db}/table/{table}/preview`	Devuelve hasta 50 filas de una tabla.
POST	`/api/translate`	Traduce `{ expression, database }` y ejecuta si es posible.
POST	`/api/history/clear`	Limpia el historial de variables.

Ejemplo de solicitud de traducción:

```
{
  "expression": "σ(nota_media >= 9)(alumnos)",
  "database": "universidad_escom"
}
```

Ejemplo de respuesta exitosa (resumida):

```
{
  "status": "success",
  "sql": "SELECT * FROM alumnos WHERE (nota_media >= 9)",
  "tree": "Seleccion (σ)...",
  "tokens": [ ... ],
  "execution": {
    "columns": ["boleta", "nombre", "..."],
    "rows": [ ... ],
    "rowCount": 3,
    "elapsedMs": 4
  }
}
```

7.7. Estructura del proyecto

```
Proyecto_TT1/
├── pom.xml
├── docs/
│   ├── manual_usuario/
│   │   ├── manual_usuario.md
│   │   ├── manual_usuario.docx
│   │   ├── manual_usuario.odt
│   │   └── capturas/
│   └── ejemplos/
│       └── universidad_escom.sql
├── src/
│   ├── main/
│   │   ├── cup/parser.cup
│   │   ├── jflex/Lexer.flex
│   │   ├── java/com/ipn/escom/tt1/
│   │   │   ├── ast/            # Nodos del árbol y generación de SQL
│   │   │   ├── controller/     # API REST
│   │   │   ├── parser/         # Lexer/parser generados y excepciones
│   │   │   ├── service/        # Traducción y base de datos
│   │   │   └── Tt1AlgebraRelacionalApplication.java
│   │   └── resources/
│   │       ├── application.properties
│   │       └── static/         # HTML, CSS, JS, imágenes
│   └── test/
└── target/                     # Artefactos de compilación
```

7.8. Cómo extender la gramática

Si en TT2 necesitas agregar un operador:

1. Agrega el token en `src/main/jflex/Lexer.flex` (símbolo Unicode y palabra alternativa).
2. Declara el terminal en `src/main/cup/parser.cup`.
3. Agrega la producción y la precedencia correspondiente.
4. Crea un nodo en `ast/` que implemente `toSQL()` y `toTreeString()`.
5. Agrega la descripción del token en `TranslationService.describe`.
6. Agrega el operador al arreglo `AR_OPERATORS` de `editor.v4.js` para que aparezca en los botones y tooltips.
7. Recompila: JFlex y CUP generan las clases en `target/generated-sources`.

8. Buenas prácticas, limitaciones y solución de problemas

8.1. Buenas prácticas con la base de datos

- Usa una base de datos de práctica, no una de producción.
- Crea un usuario de solo lectura para la aplicación.
- Haz respaldos antes de experimentar con datos reales.
- Usa nombres descriptivos y evita palabras reservadas.
- Define claves primarias y foráneas; el diccionario las mostrará.
- Agrega comentarios a tablas y columnas; el diccionario los muestra y son útiles en clase.
- Mantén los tipos consistentes: no compares una fecha con un número ni una cadena con un entero si puedes evitarlo.

8.2. Buenas prácticas al escribir expresiones

- Escribe una operación a la vez y prueba con **Traducir y ejecutar**.
- Usa paréntesis para explicitar la precedencia.
- Prefiere `==` para igualdad y `<-` para asignación.
- Proyecta antes de reunir si las tablas comparten nombres de columna.
- Usa `COUNT(*)` para contar filas y `COUNT(columna)` para contar valores no nulos.
- Revisa la tokenización cuando un nombre se comporta de forma extraña.
- Guarda relaciones intermedias en `@1`, `@2`, … para consultas largas.
- No dependas de que SQL elimine duplicados en una proyección.
- Prueba la consulta en la base de datos correcta.

8.3. Limitaciones conocidas

Limitación	Efecto	Alternativa
Solo lectura	No se pueden modificar datos.	Usa MySQL directamente para cambios.
Sin diagramas E-R ni modelo relacional	No se generan diagramas.	Se implementará en TT2.
Una sola conexión en memoria	Todos los usuarios de la instancia comparten la conexión.	Uso local o un servidor por usuario.
Credenciales en memoria	Se pierden al reiniciar.	Vuelve a conectar.
Límite de 500 filas	Los resultados se truncan.	Filtra o agrega en SQL.
Tiempo máximo de 10 s	Consultas lentas se cancelan.	Optimiza o divide la consulta.
Vista previa de 50 filas	No muestra toda la tabla.	Usa una consulta con la interfaz o MySQL.
Sin `DISTINCT` automático	La proyección puede devolver duplicados.	Agrega `DISTINCT` en SQL si lo necesitas.
Sin alias en AR	No puedes renombrar columnas en la expresión.	Renombra en SQL o en la tabla.
Sin `ORDER BY`, `LIMIT`, `HAVING`	No se puede ordenar, paginar ni filtrar grupos.	Usa SQL directamente.
Sin reuniones externas	No hay `LEFT/RIGHT/FULL JOIN`.	Usa SQL directamente.

Limitación	Efecto	Alternativa
Sin división relacional	No se puede expresar la división.	Implementa la división con diferencia y producto.
Columnas duplicadas en resultados	Una columna puede sobrescribir a otra.	Proyecta nombres distintos.
Dependencia de CDN	Sin internet se pierde el editor enriquecido.	Usa el área de texto de respaldo.
Diferencias MySQL/MariaDB	`EXCEPT` y otras funciones pueden variar.	Revisa la versión del servidor.
Colisiones con palabras reservadas	Un nombre como `promedio` se interpreta como `PROM`.	Cualifica o renombra.

8.4. Preguntas frecuentes

¿Necesito crear una base de datos dentro del sistema? No. El sistema se conecta a bases que ya existen en tu MySQL. Para crear tablas y datos usa MySQL Workbench, la consola `mysql` o el script de demostración.

¿Dónde se guardan mis contraseñas? En la memoria del servidor mientras la aplicación está en ejecución. No se escriben en archivos ni en la base de datos. Al reiniciar, debes volver a conectar.

¿Puedo usar MariaDB? Sí, el proyecto se probó con MariaDB 11.8 y el controlador JDBC de MySQL. Algunas diferencias de versión pueden afectar `EXCEPT` o funciones específicas.

¿Por qué no aparece mi base de datos? El sistema excluye las bases internas (`information_schema`, `mysql`, `performance_schema`, `sys`). Si tu base no aparece, revisa que exista, que el usuario tenga permisos y que hayas actualizado la conexión.

¿Por qué el resultado está vacío? Puede que la condición no coincida con ninguna fila, que la tabla esté vacía o que la reunión natural esté igualando columnas de más. Revisa el SQL y prueba la consulta directamente en MySQL.

¿Por qué el árbol se corta? El panel tiene una altura máxima y desplazamiento. Desplázate dentro del recuadro del árbol para ver el resto.

¿Puedo copiar el SQL generado? Sí. Selecciónalo en el panel **SQL generado** o en **Sentencia SQL completa** y cópialo. Ten en cuenta que los alias `_q...` son internos y cambian entre ejecuciones.

¿Cómo sé si el sistema está usando mi base y no otra? Revisa el selector **Base de datos** en el panel de conexión. El diccionario y las consultas usan esa base.

¿Puedo trabajar sin internet? La traducción funciona, pero Monaco y los estilos se cargan desde CDN. Sin internet, el editor usa un área de texto simple. Para una demostración en un aula sin red, prueba antes y considera descargar los recursos localmente.

8.5. Seguridad y privacidad

- Usa credenciales de un usuario de solo lectura.
- No ejecutes el servidor en una red pública sin autenticación: la API no tiene login.
- No compartas capturas con contraseñas o datos personales reales.
- La aplicación no está diseñada para producción ni para múltiples usuarios simultáneos.
- El sistema valida que la consulta empiece con `SELECT` o `(`, pero esa validación no sustituye a los permisos de MySQL. La defensa real es un usuario con privilegios mínimos.

- Los datos de las tablas se muestran en el navegador; evita usarlos en equipos compartidos si son sensibles.

8.6. Rendimiento

- El sistema limita los resultados a 500 filas y las vistas previas a 50.
- Las consultas tienen un tiempo máximo de 10 segundos.
- El producto cartesiano puede crecer muy rápido: si `alumnos` tiene 10 filas y `carreras` 4, el producto tiene 40; con tablas grandes puede ser inviable.
- Las subconsultas generadas no están optimizadas. Para consultas grandes, prueba el SQL en MySQL y analiza el plan con `EXPLAIN`.
- El diccionario consulta `information_schema`; en bases con cientos de tablas puede tardar unos segundos.

8.7. Lista de verificación para una demostración

- ☐ MySQL está iniciado y responde.
- ☐ La base de demostración está cargada.
- ☐ La aplicación está corriendo en el puerto correcto.
- ☐ El navegador tiene acceso a los CDN (o se probó el respaldo).
- ☐ La conexión funciona con el usuario previsto.
- ☐ El diccionario muestra las tablas y claves.
- ☐ Una consulta simple se traduce y ejecuta.
- ☐ Una consulta con reunión y una con agrupación funcionan.
- ☐ El historial guarda y recupera `@1`.
- ☐ Se preparó un ejemplo de error para explicar el catálogo.
- ☐ Las capturas y datos de la presentación no contienen información sensible.

9. Glosario

Álgebra relacional. Lenguaje formal de operaciones sobre relaciones que sirve como base teórica de SQL.

Atributo. Columna de una relación; tiene nombre y dominio.

AST (árbol de sintaxis abstracta). Representación en árbol de una expresión, usada para traducir a SQL.

Base de datos. Colección organizada de datos y de metadatos.

Clave foránea (FK). Atributo que referencia la clave primaria de otra tabla.

Clave primaria (PK). Atributo o conjunto de atributos que identifica de forma única cada fila.

Columna. Ver atributo.

DBMS. Sistema gestor de bases de datos.

Diferencia (−). Operación de conjuntos que devuelve las tuplas de la primera relación que no están en la segunda.

Dominio. Conjunto de valores válidos para un atributo.

Esquema. Definición de las relaciones, atributos y restricciones de una base de datos.

Expresión. Combinación de operadores, relaciones, atributos y literales que produce una relación o un valor.

Función de agregación. Función que resume un conjunto de valores: `SUM`, `AVG`, `MAX`, `MIN`, `COUNT`.

Interrupción (token). Unidad léxica reconocida por el analizador.

JDBC. API de Java para conectarse a bases de datos relacionales.

Lexema. Texto exacto de un token.

Monaco Editor. Componente de edición de código usado por la interfaz.

MySQL. Sistema gestor de bases de datos relacional.

NULL. Valor especial que representa ausencia de dato.

Parser. Analizador sintáctico que verifica la gramática y construye el árbol.

Producto cartesiano (×). Operación que combina cada tupla de una relación con cada tupla de otra.

Proyección (π). Operación que selecciona atributos.

Relación. Tabla con esquema y tuplas.

Reunión (⋈). Operación que combina tuplas de dos relaciones según una condición.

Selección (σ). Operación que filtra tuplas según una condición.

SQL. Lenguaje estructurado de consulta usado por los gestores relacionales.

Token. Ver interrupción.

Tupla. Fila de una relación.

Unión (∪). Operación que devuelve las tuplas presentes en cualquiera de dos relaciones compatibles.

Variable de historial. Nombre `@n` que guarda una relación para reutilizarla.

10. Anexos

Anexo A. Base de datos de demostración

El script `docs/ejemplos/universidad_escom.sql` crea la base `universidad_escom`. Estas son sus tablas:

carreras (4 filas)

Columna	Tipo	Restricción
`id_carrera`	`INT`	PK, autoincremental
`nombre`	`VARCHAR(100)`	NOT NULL
`area`	`VARCHAR(50)`	NOT NULL
`duracion_semestres`	`INT`	NOT NULL, default 8

alumnos (10 filas)

Columna	Tipo	Restricción
`boleta`	`CHAR(10)`	PK
`nombre`	`VARCHAR(60)`	NOT NULL
`apellido_paterno`	`VARCHAR(60)`	NOT NULL
`apellido_materno`	`VARCHAR(60)`	NULL
`fecha_nacimiento`	`DATE`	NULL
`correo`	`VARCHAR(120)`	UNIQUE
`telefono`	`VARCHAR(15)`	NULL
`id_carrera`	`INT`	FK → `carreras.id_carrera`
`semestre`	`INT`	NOT NULL, default 1
`nota_media`	`DECIMAL(4,2)`	default 0.00
`estatus`	`VARCHAR(20)`	NOT NULL, default `Activo`

profesores (4 filas)

Columna	Tipo	Restricción
`id_profesor`	`INT`	PK, autoincremental
`nombre`	`VARCHAR(100)`	NOT NULL
`correo`	`VARCHAR(120)`	NULL
`departamento`	`VARCHAR(80)`	NULL

materias (6 filas)

Columna	Tipo	Restricción
`id_materia`	`INT`	PK, autoincremental
`nombre`	`VARCHAR(100)`	NOT NULL
`creditos`	`INT`	NOT NULL
`semestre`	`INT`	NOT NULL
`id_profesor`	`INT`	FK → `profesores.id_profesor`

inscripciones (18 filas)

Columna	Tipo	Restricción
id_inscripcion	INT	PK, autoincremental
boleta	CHAR(10)	FK → alumnos.boleta
id_materia	INT	FK → materias.id_materia
periodo	VARCHAR(10)	NOT NULL
calificacion	DECIMAL(4,2)	NULL

pagos (7 filas)

Columna	Tipo	Restricción
id_pago	INT	PK, autoincremental
boleta	CHAR(10)	FK → alumnos.boleta
concepto	VARCHAR(80)	NOT NULL
monto	DECIMAL(10,2)	NOT NULL
fecha_pago	DATE	NULL
estado	VARCHAR(20)	NOT NULL, default Pendiente

Anexo B. Tarjeta de referencia rápida

Operador	Símbolo	Alternativas	Ejemplo
Selección	σ	SELEC, SIGMA	σ(edad > 18)(alumnos)
Proyección	π	PROY, PI	π(nombre)(alumnos)
Reunión natural	⋈	JOIN	alumnos ⋈ carreras
Reunión theta	—	JOIN	JOIN(alumnos, carreras, alumnos.id_carrera == carreras.id_carrera)
Unión	∪	UNION	A ∪ B
Intersección	∩	INTERSEC, INTER	A ∩ B
Diferencia	−	DIF, -	A − B
Producto	×	PRODCART, *	A × B
Agrupación	γ	AGRUPAR, GROUP	γ(id_carrera; COUNT(*))(alumnos)
Suma	Σ	SUM, SUMA	SUM(creditos)
Promedio	—	AVG, PROM, PROMEDIO	AVG(nota_media)
Máximo	—	MAX	MAX(nota_media)
Mínimo	—	MIN	MIN(nota_media)
Conteo	—	COUNT, CONTAR	COUNT(*)
Asignación	—	<-, =	@1 <- σ(...)(alumnos)

Condiciones: ==, !=, >, >=, <, <=; combinaciones con &&, ||, !.

Funciones: sin, cos, tan, asin, acos, atan, atan2, sqrt, raiz, log, ln, exp, abs, floor, ceil, round, mod, pow, radians, degrees.

Anexo C. Banco de ejercicios

Resuelve cada ejercicio con una expresión de álgebra relacional y verifica el SQL generado. Las soluciones están al final del anexo.

1. Muestra todos los datos de los alumnos.
2. Muestra únicamente `boleta`, `nombre` y `nota_media` de los alumnos.
3. Muestra los alumnos con `nota_media` mayor o igual a 9.
4. Muestra nombre y estatus de los alumnos activos.
5. Muestra nombre y nota media de los alumnos de la carrera 1 con nota mayor o igual a 8.
6. Muestra nombre y área de todos los alumnos, usando una reunión con carreras.
7. Calcula el número de alumnos por carrera.
8. Calcula el promedio de nota media por carrera.
9. Muestra las materias con más de 6 créditos.
10. Muestra nombre y boleta de los alumnos nacidos a partir del 1 de enero de 2005.
11. Muestra los nombres de alumnos y profesores usando una unión.
12. Muestra los nombres de alumnos que no aparecen como nombres de profesores, usando diferencia.
13. Calcula el número de alumnos y la nota máxima por carrera.
14. Muestra los alumnos cuyo seno de `nota_media` sea mayor que 0.5.
15. Muestra nombre y nota media multiplicada por 1.1.
16. Guarda en `@1` los alumnos con nota mayor o igual a 8 y luego muestra solo nombre y nota media.
17. Provoca un error sintáctico, identifica el código y corrígelo.
18. Provoca un error léxico, identifica la línea y la columna, y corrígelo.

Soluciones

1. `alumnos`
2. `π(boleta, nombre, nota_media)(alumnos)`
3. `σ(nota_media >= 9)(alumnos)`
4. `π(nombre, estatus)(σ(estatus == "Activo")(alumnos))`
5. `π(nombre, nota_media)(σ(id_carrera == 1 && nota_media >= 8)(alumnos))`
6. `π(nombre, area)(alumnos ⋈ π(id_carrera, area)(carreras))`
7. `γ(id_carrera; COUNT(*))(alumnos)`
8. `γ(id_carrera; AVG(nota_media))(alumnos)`
9. `σ(creditos > 6)(materias)`
10. `π(nombre, boleta)(σ(fecha_nacimiento >= "2005-01-01")(alumnos))`
11. `π(nombre)(alumnos) ∪ π(nombre)(profesores)`
12. `π(nombre)(alumnos) − π(nombre)(profesores)`
13. `γ(id_carrera; COUNT(*), MAX(nota_media))(alumnos)`
14. `π(nombre)(σ(sin(nota_media) > 0.5)(alumnos))`
15. `π(nombre, nota_media * 1.1)(alumnos)`
16. `@1 <- σ(nota_media >= 8)(alumnos)` y después `π(nombre, nota_media)(@1)`
17. Por ejemplo, `σ(nota_media >= 9)(alumnos` produce `SIN-01`; se corrige cerrando el paréntesis.
18. Por ejemplo, `σ(nota_media >= 9)(alumnos) ?` produce `LEX-01` en la línea 1, columna 29; se corrige eliminando `?`.

Anexo D. Referencias recomendadas

- Elmasri, R. y Navathe, S. B. *Fundamentals of Database Systems*. Pearson. Capítulos de modelo relacional y álgebra relacional.
- Silberschatz, A., Korth, H. F. y Sudarshan, S. *Database System Concepts*. McGraw-Hill. Capítulo de modelo relacional.
- Documentación oficial de MySQL 8.0: https://dev.mysql.com/doc/
- Documentación de MariaDB: https://mariadb.com/kb/en/documentation/
- Documentación de Spring Boot: https://spring.io/projects/spring-boot
- Documentación de JFlex: https://jflex.de/
- Documentación de Java CUP: http://www2.cs.tum.edu/projects/cup/
- Documentación de Monaco Editor: https://microsoft.github.io/monaco-editor/